# BMP Compliance Platform

## Project Technical Guide

**Client:** BMP & Co. LLP · **Application version:** 1.0.0 · **Guide date:** 1 October 2026

| | |
|---|---|
| **What this guide covers** | How to run and use the project · project structure · the back end (API files, settings, database integration) · the front end · pipelines · the important core-logic files · a ready-made Postman collection that tests every API |
| **State of the project described** | The *Latest_Implementation_with_approved_Database* folder: approved Phase 1 PostgreSQL database (unchanged) + FastAPI back end + the approved dashboard look-and-feel as a web front end |
| **Verification** | 307 automated tests pass. The Postman collection in this guide (114 requests, 216 checks) was run with Newman against a freshly built database: **0 failures**, and it re-runs cleanly on the same database |
| **Companion documents** | `BMP_UI_Backend_Implementation_Guide.md` (product view and demo script) · `BMP_UI_Backend_Implementation_Log.md` (what was built, found and fixed) · `BMP_Phase1_Database_Guide.md`, `docs/ERD.md`, `docs/DATA_DICTIONARY.md` (the database) |

Contents (click an entry to jump to it)


**PART 1 — Orientation**
1. The project in one page

**PART 2 — Commands to run and use the project**
2. Set-up, run and use

**PART 3 — Project structure**
3. Folders and files

**PART 4 — The back end**
4. Back-end design

**PART 5 — The front end**
5. Front-end design

**PART 6 — Pipelines**
6. Pipelines

**PART 7 — Important core-logic files**
7. Core logic, file by file

**PART 8 — Testing the API with Postman**
8. The Postman collection

**PART 9 — Quick answers**
9. Frequently asked questions

**Appendix A — Postman environment (JSON)**

**Appendix B — Postman collection (JSON)**

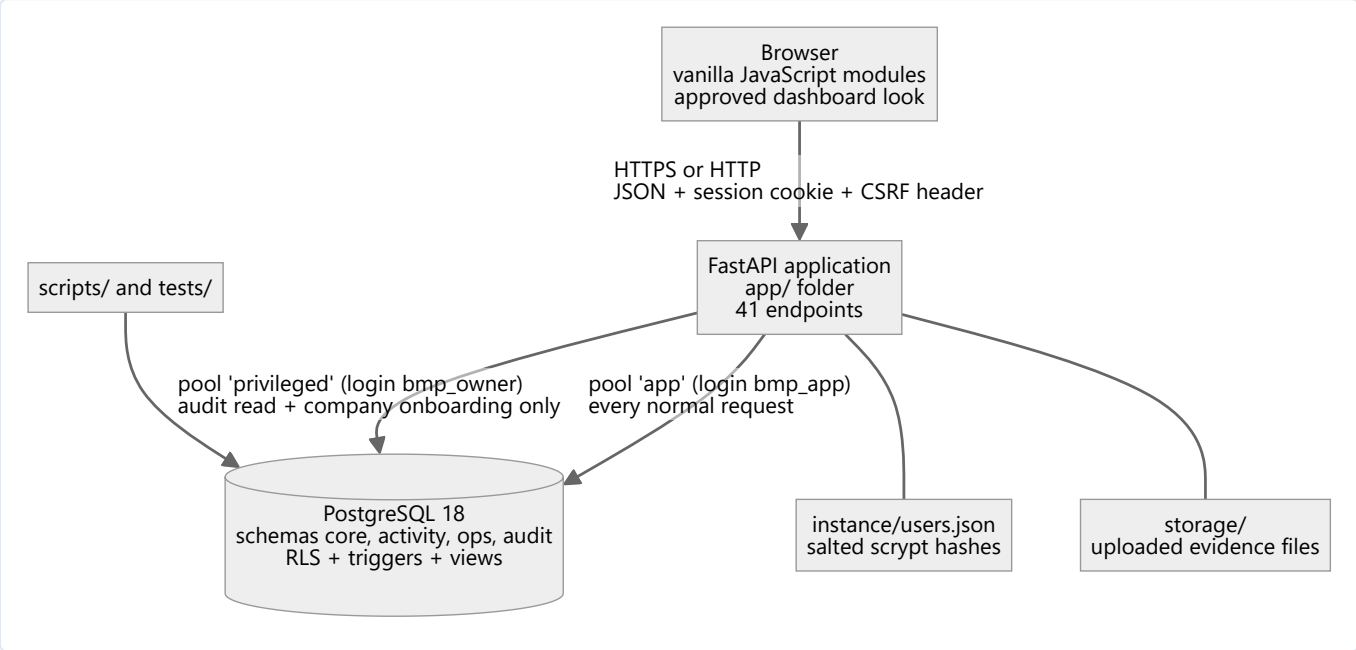
**Appendix C — Glossary**

# PART 1 — Orientation

## 1. The project in one page

The BMP Compliance Platform is the **system of record** for a firm of company secretaries and chartered accountants: it stores each client company's statutory data (directors, shareholders, capital, meetings, filings, rules) and runs compliance **activities** (for example "hold a Board Meeting") as ordered, dated, evidence-backed tasks.

It has three layers. The database is the authority; everything above it is deliberately thin.




| Layer | What it is | Size |
|---|---|---|
| **Database** (unchanged Phase 1) | 11 migrations, 5 seed files; 66 base tables in four schemas (core 55, activity 6, ops 4, audit 1); 13 calculated views; row-level security on 32 company tables; audit trigger; no-delete triggers; two login roles | `db/` |
| **Back end** | FastAPI app with a **metadata-driven** generic record engine: it reads the live database catalog, so every table and column is reachable without hand-written code per table | 43 routes (41 API endpoints, a health probe, the page), 65 record types, 13 calculated views |
| **Front end** | The approved dashboard theme, served as static files; 17 ES-module scripts, no build step, no internet needed | 1 HTML template, 2 CSS files, Bootstrap vendor files |
| **Quality** | 19 database tests + 288 web tests; a Postman collection | 307 tests pass in about 20 seconds |

### 1.1 Technology stack (versions this guide was verified with)

| Component | Version | Used for |
|---|---|---|
| Python | 3.12 (3.10+ works) | Everything outside the database |
| PostgreSQL + pgvector | 18 + 0.8.6 (16+ works) | The database; `vector` type for future document search |
| FastAPI / Starlette / Uvicorn | 0.142.2 / 1.7.0 / 0.54.0 | Web framework, routing, middleware, sessions; the server |
| psycopg / psycopg-pool | 3.3.6 / 3.3.3 | PostgreSQL driver and connection pools |

Also used: Pydantic 2.13.5 (sign-in and password bodies), Jinja2 3.1.6 (the one HTML page), itsdangerous 2.2.0 (signed session cookie), python-multipart 0.0.32 (uploads), openpyxl 3.1.5 (Excel export), python-dotenv 1.2.3 (`.env`), pytest 9.1.1 and httpx 0.28.1 (tests), Bootstrap 5.3.3 (vendored under `app/static/vendor`, no CDN).

# PART 2 — Commands to run and use the project

All commands are for **Windows PowerShell**, run from the project folder (the one that contains `app`, `db`, `scripts` and `tests`). macOS/Linux users replace `.\venv\Scripts\Activate.ps1` with `source venv/bin/activate`; everything else is identical.

## 2. Set-up, run and use

> The three demo sign-ins are `admin@bmpco.in` (all five companies, administrator), `manager@bmpco.in` (three companies, can edit) and `client@nexatech.co.in` (Nexatech only, read-only). All data is fictional.

### 2.1 Prerequisites (install once)

| Needed | Version | Note |
|---|---|---|
| PostgreSQL | 16 or newer | Remember the `postgres` administrator password |
| pgvector | to match PostgreSQL | Not bundled; see `BMP_Phase1_Database_Guide.md` §11 |
| Python | 3.10 or newer | Tick *Add Python to PATH* |
| Browser | Chrome, Edge, Firefox | |
| Node.js (optional) | 18+ | Only for running the Postman collection from the command line with Newman |

### 2.2 First-time set-up

```
cd "C:\Users\<you>\Desktop\BMP\Latest_Implementation_with_approved_Database"

# 1. A private Python environment (delete the 'venv' folder first if it arrived from another computer)
python -m venv venv
.\venv\Scripts\Activate.ps1            # if blocked: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
pip install -r requirements.txt

# 2. Settings: copy the template, then edit .env in a text editor (see section 4.3)
copy .env.example .env

# 3. Build and fill the database (creates roles, extensions, 11 migrations, 5 seed files)
python scripts/setup_db.py --seed

# 4. Give the three demo users a sign-in password (asked twice, not shown)
python scripts/manage_users.py init-demo

# 5. Check everything works (about 20 seconds; expect: 307 passed)
python -m pytest

# 6. Start the application
python scripts/run_server.py
```

Open **http://127.0.0.1:8000** and sign in. Stop the server with **Ctrl + C**.

> **Type the database lines into `.env`, not into the terminal.** At minimum `.env` needs `DATABASE_URL` (login `bmp_owner`), `DATABASE_URL_APP` (login `bmp_app`) and `DATABASE_URL_ADMIN` (the PostgreSQL administrator, used only to create the database and by the tests), plus `APP_SECRET_KEY`. Generate a secret with `python -c "import secrets; print(secrets.token_urlsafe(48))"`.

## 2.3 Everyday commands

| Task | Command | Notes |
|---|---|---|
| Activate the environment | `.\venv\Scripts\Activate.ps1` | Each new terminal |
| Start the app | `python scripts/run_server.py` | `http://127.0.0.1:8000` |
| Start on another port | `python scripts/run_server.py --port 8080` | If 8000 is busy |
| Let other computers connect | `python scripts/run_server.py --host 0.0.0.0` | Use HTTPS behind a reverse proxy in production |
| Auto-restart on code change | `python scripts/run_server.py --reload` | Development only |
| Several worker processes | `python scripts/run_server.py --workers 2` | Production; not combined with `--reload` |
| Same as the script, by hand | `uvicorn app.main:app --host 127.0.0.1 --port 8000` | `app.main` builds the app lazily |
| Interactive API documentation | open `http://127.0.0.1:8000/api/docs` | Development only (hidden when `APP_ENV=production`) |
| Health check | `curl http://127.0.0.1:8000/api/health` | Returns `{"status":"ok","database":true,...}` |

## 2.4 Database commands

| Task | Command |
|---|---|
| Create database, roles, apply migrations and seed data | `python scripts/setup_db.py --seed` |
| Apply only new migrations (safe to repeat; applied files are skipped; add `--seed` to also run any new seed files) | `python scripts/setup_db.py` |
| **Delete** and rebuild the database (development only) | `python scripts/setup_db.py --reset --seed` |
| Run the Phase 1 database demonstration (10 numbered scenes) | `python scripts/run_demo.py` |
| Open the demo SQL queries | `demo/demo_queries.sql` (run in any SQL client) |
| Back up the database | `pg_dump -U bmp_owner -F c -f bmp.backup bmp` — **and** copy the `storage/` folder |

## 2.5 Users and passwords

The database deliberately has **no password column**. Passwords are salted scrypt hashes in `instance/users.json`. A person can sign in only when they have **both** a hash there **and** an active row in `core.app_user` (which gives them a role and company access).

| Task | Command |
|---|---|
| Show who exists and who has a password | `python scripts/manage_users.py list` |
| Set one person's password | `python scripts/manage_users.py set-password someone@firm.in` |
| One password for the three demo users (development only) | `python scripts/manage_users.py init-demo` |
| Change my own password | In the app: user menu → *Change my password* |
| Administrator sets someone else's | In the app: Administration → Users → open the person → *Set password* |

New people are added in the app under **Administration** → **Users** and **User company access**.

## 2.6 Test commands

| What | Command | Expected |
|---|---|---|
| Everything | `python -m pytest` | `307 passed` |
| Only the web-application tests | `python -m pytest tests/api` | 288 passed |
| Only the Phase 1 database tests | `python -m pytest tests --ignore=tests/api` | 19 passed |
| One file, quiet | `python -m pytest tests/api/test_workflows.py -q` | 35 passed |
| One test by name | `python -m pytest -k "csrf" -v` | |
| List tests without running | `python -m pytest --collect-only -q` | |
| Verify requirement traceability | `python scripts/check_traceability.py` | Exit code 0 = every spreadsheet row is mapped |
| Regenerate the traceability document | `python scripts/generate_traceability.py` | Rewrites `docs/FIELD_TRACEABILITY.md` |

The web tests build and delete a throw-away database `bmp_api_test` (this is why `DATABASE_URL_ADMIN` is needed). Your real `bmp` data is never touched.

## 2.7 Using the API without the screen (curl)

Sign in once, keep the cookie, and send the CSRF token on every change. (Replace the password placeholder; in PowerShell use `curl.exe`, not the `curl` alias.)

```
# sign in: the response contains "csrf"; the cookie is saved in cj.txt
curl -s -c cj.txt -H "Content-Type: application/json" \
     -d '{"email":"admin@bmpco.in","password":"<your password>"}' http://127.0.0.1:8000/api/auth/login

# read (no CSRF header needed)
curl -s -b cj.txt http://127.0.0.1:8000/api/companies

# change (CSRF header required; value comes from the login response)
curl -s -b cj.txt -H "X-CSRF-Token: <csrf from login>" -H "Content-Type: application/json" \
     -d '{"company_id":2,"name":"Test RTA","valid_from":"2024-04-01"}' \
     http://127.0.0.1:8000/api/entities/company_rta
```

For a complete, assertion-checked walk through every endpoint, use the Postman collection in Part 7.

## 2.8 Running the Postman collection from the command line

```
npm install -g newman
cd docs\postman
newman run BMP_Compliance_Platform.postman_collection.json `
       -e BMP_Compliance_Platform.postman_environment.json `
       --env-var adminPassword=<pw> --env-var managerPassword=<pw> --env-var clientPassword=<pw> `
       --working-dir .
```

## 2.9 Command cheat-sheet

| I want to... | Run |
|---|---|
| Start working | `.\venv\Scripts\Activate.ps1` then `python scripts/run_server.py` |
| Start from a clean demo database | `python scripts/setup_db.py --reset --seed` then `python scripts/manage_users.py init-demo` |
| Prove nothing is broken | `python -m pytest` |
| Show the database features in a terminal | `python scripts/run_demo.py` |

| I want to… | Run |
|---|---|
| Test every API | Import the two Postman files, or run Newman (2.8) |
| Add a database change | Add `db/migrations/012_….sql`, then `python scripts/setup_db.py` |
| Package up for another computer | Copy the folder **without** `venv`, `.env`, `instance`, `storage`, `tmp`; recreate them there |

## 2.10 Common problems

| Symptom | Cause and fix |
|---|---|
| *The term 'DATABASE_URL=…' is not recognized* | Those lines belong inside the `.env` file, not the terminal |
| `password authentication failed for user "postgres"` | Wrong password in `DATABASE_URL_ADMIN` |
| `ModuleNotFoundError: fastapi` / `psycopg_pool` | Environment not active, or `pip install -r requirements.txt` was skipped |
| *Email or password is incorrect* | No password yet (`manage_users.py list`), user inactive, or e-mail differs from `core.app_user` |
| *Too many failed sign-in attempts* | 5 failures in 10 minutes lock that e-mail + address pair; wait, or restart the server |
| Signed out at every server restart | `APP_SECRET_KEY` is empty (a temporary key is invented); set it |
| Port 8000 busy | `--port 8080` |
| Tests say *DATABASE_URL_ADMIN … must be set* | Add it to `.env` |
| Upload refused | Allowed types only; 25 MB maximum (`MAX_UPLOAD_MB`) |
| *This record was changed by someone else* | Reopen the record, then repeat your edit |
| *File name too long* errors when `STORAGE_DIR` is deep inside nested folders (Windows 260-character limit) | Point `STORAGE_DIR` at a short path such as `C:\bmp_storage` |

# PART 3 — Project structure

## 3. Folders and files

### 3.1 Directory tree

```
Latest_Implementation_with_approved_Database/
│
├── app/                          THE WEB APPLICATION (back end + front end)
│   ├── main.py                   application factory, middleware, error handlers, page + health routes
│   ├── config.py                 settings read from .env (one frozen Settings object)
│   ├── db.py                     connection pools and the per-request transaction
│   ├── deps.py                   FastAPI dependencies: signed-in user, admin check, DB sessions
│   ├── access.py                 UserCtx and Scope: who may see / change which rows
│   ├── schema_meta.py            reads the live database catalog (columns, keys, enums, lookups)
│   ├── registry.py               navigation, labels, write rules and scope per table, view list
│   ├── crud.py                   Engine: generic list / get / create / update / archive / options / views
│   ├── coerce.py                 JSON value -> typed database value
│   ├── errors.py                 ApiError and translation of PostgreSQL errors to messages
│   ├── security.py               scrypt passwords, credentials file, login throttle, CSRF
│   ├── responses.py              exact-value JSON response class
│   ├── routers/                  THE API (one file per area)
│   │   ├── auth.py               sign in / out, who am I, passwords
│   │   ├── meta.py               /api/meta and dropdown options
│   │   ├── entities.py           generic CRUD for every table + history + CSV
│   │   ├── companies.py          list, onboarding, years, overview, JSON/Excel export
│   │   ├── people.py             directors and KMP register, add-to-board in one step
│   │   ├── meetings.py           meetings book, meeting detail, attendance by roster
│   │   ├── views.py              calculated views and applicability rules
│   │   ├── activities.py         templates, start activity, task workflow, confirmations
│   │   ├── documents.py          evidence upload (hashed, versioned) and download
│   │   └── audit.py              the Activity Log (reads the audit table)
│   ├── templates/
│   │   └── dashboard.html        the single HTML page (login + application shell)
│   └── static/
│       ├── stylesheets/          dashboard.css (approved theme, unchanged) + app.css (additions)
│       ├── vendor/               Bootstrap CSS + JS (no CDN)
│       └── scripts/              17 ES modules: main, api, state, form, entity, and one per screen
│
├── db/                           THE APPROVED DATABASE (unchanged)
│   ├── migrations/               001 … 011  schema, functions, views, triggers, RLS, grants
│   └── seed/                     01 … 05    lookups, rules, users, companies, people, capital, meetings
│
├── scripts/                      COMMAND-LINE TOOLS
│   ├── setup_db.py               create database/roles/extensions, apply migrations and seeds
│   ├── run_server.py             start the web application (uvicorn)
│   ├── manage_users.py           list users / set passwords
│   ├── run_demo.py               10-scene database demonstration
│   ├── check_traceability.py     every spreadsheet row mapped?  (exit code)
│   └── generate_traceability.py  regenerate docs/FIELD_TRACEABILITY.md
│
├── tests/                        307 AUTOMATED TESTS
│   ├── conftest.py, test_*.py    19 Phase 1 database tests
│   └── api/                      288 web-application tests (conftest, payloads, 6 test files)
│
├── docs/                         ERD.md, DATA_DICTIONARY.md, FIELD_TRACEABILITY.md,
│   └── postman/                  the Postman collection + environment + sample upload file
├── demo/demo_queries.sql         SQL used in the database demonstration
├── README.md                     quick start
├── requirements.txt              Python packages
├── .env.example  /  .env         settings template / your real settings (never commit .env)
├── .gitignore
├── BMP_UI_Backend_Implementation_Guide.md / _Log.md, BMP_Phase1_Database_Guide.md
├── BMP_Project_Requirements_v1_0.docx, BMP_Technical_Overview_v1.1.docx,
│   BMP_Master Checklist_Dashboard Requirements.xlsx          (requirements sources)
│
└── created while running (not part of the source; all git-ignored)
    ├── venv/                     your Python environment
```

```
├── instance/users.json       sign-in password hashes
├── storage/                  uploaded evidence files (<company_id>/<sha-prefix>/<sha>_<name>)
└── tmp/ , .pytest_cache/     scratch
```

## 3.2 What each top-level folder is for

| Folder | Purpose | Edit it when… |
|---|---|---|
| `app/` | The product: API and UI | Changing behaviour, screens, rules about access |
| `db/` | The approved database | **Only by adding a new numbered migration.** The approved migrations 001–011 are not edited |
| `scripts/` | Operations tools | Changing set-up or demo behaviour |
| `tests/` | Safety net | Whenever behaviour changes |
| `docs/` | Documentation and the Postman files | Updating documentation |
| `instance/` , `storage/` | Runtime data (credentials, uploaded files) | Never by hand; back up with the database |

## 3.3 File inventory (every source file, with size and purpose)

### Back end — support modules

| File | Lines | Purpose |
|---|---|---|
| `app/main.py` | 163 | Application factory, lifespan (pools, catalog), middleware, error handlers, page and health routes |
| `app/config.py` | 117 | `Settings` read once from `.env` / environment; start-up safety checks |
| `app/db.py` | 51 | Connection pools and the per-request transaction that sets `app.current_user_id` |
| `app/deps.py` | 63 | Dependencies: `current_user` (session + CSRF), `admin_user` , `session` , `privileged_session` , company checks |
| `app/access.py` | 166 | `UserCtx` , write rules per table, `Scope` SQL builder (who may see / change which rows) |
| `app/schema_meta.py` | 225 | Reads the live PostgreSQL catalog: columns, kinds, keys, enums, lookups |
| `app/registry.py` | 433 | Navigation, labels, list/search columns, children, sections, write mode, scope, view list |
| `app/crud.py` | 473 | `Engine` : generic list / get / options / create / update / archive / view rows |
| `app/coerce.py` | 106 | Converts JSON values into typed database values |
| `app/errors.py` | 107 | `ApiError` ; translation of PostgreSQL errors into HTTP status + plain-language message |
| `app/security.py` | 114 | scrypt passwords, credentials file, login throttle, CSRF compare |
| `app/responses.py` | 32 | JSON response class that keeps numbers and dates exact |
| `app/__init__.py` | 1 | Package marker |

### Back end — API routers

| File | Lines | Purpose |
|---|---|---|
| `app/routers/auth.py` | 101 | Sign in / out, `me` , change password, admin set password |
| `app/routers/meta.py` | 159 | `/api/meta` (self-describing API) and `/api/options/{entity}` |
| `app/routers/entities.py` | 189 | Generic CRUD for every table, archive/restore, history, CSV export |
| `app/routers/companies.py` | 238 | Company list, onboarding, years, overview, JSON and Excel export |
| `app/routers/people.py` | 83 | Directors & KMP register; add a person to a board in one step |
| `app/routers/meetings.py` | 113 | Meetings book, meeting detail with roster, attendance save |
| `app/routers/views.py` | 49 | Calculated views and applicability rules |

| File | Lines | Purpose |
|---|---|---|
| `app/routers/activities.py` | 315 | Templates, start activity, task and instance workflow, confirmations |
| `app/routers/documents.py` | 111 | Evidence upload (hash, version, store, link) and checked download |
| `app/routers/audit.py` | 77 | Activity Log read from `audit.log` on the privileged connection |
| `app/routers/__init__.py` | 0 | Package marker |

### Front end — page and styles

| File | Lines | Purpose |
|---|---|---|
| `app/templates/dashboard.html` | 150 | The single HTML page: sign-in card + application shell |
| `app/static/stylesheets/dashboard.css` | 601 | Approved dashboard theme (never edited) |
| `app/static/stylesheets/app.css` | 115 | Additions for the new screens (never overrides the approved theme) |
| `app/static/vendor/bootstrap.min.css` | 227 KB | Bootstrap 5.3.3 CSS (served locally, no CDN) |
| `app/static/vendor/bootstrap.bundle.min.js` | 79 KB | Bootstrap 5.3.3 JavaScript (modals, dropdowns, accordion) |

### Front end — scripts

| File | Lines | Purpose |
|---|---|---|
| `app/static/scripts/main.js` | 270 | Entry point: sign in/out, load metadata and companies, open a company, routing ( `go` ), password dialog, header buttons |
| `app/static/scripts/api.js` | 89 | The **only** network code: `get/post/patch/put/upload/download` , `ApiError` , CSRF header, 401 event, named calls |
| `app/static/scripts/state.js` | 48 | The one shared `state` object plus helpers ( `company` , `entityMeta` , `canWriteCompany` , `lookup` ) |
| `app/static/scripts/util.js` | 135 | Helpers: `$` , `esc` (HTML escaping), toast, date/number formatting, CSV, drop-zone, debounce |
| `app/static/scripts/sidebar.js` | 79 | Builds the accordion from the server's `nav` ; icons; active link; count chips |
| `app/static/scripts/form.js` | 250 | Field engine: draws a control for any column from its metadata, collects values, shows server errors |
| `app/static/scripts/entity.js` | 452 | Generic table page (search, sort, page, archive toggle, CSV) and record modal (fields, children, history, archive) |
| `app/static/scripts/companies.js` | 74 | Companies landing page and the *Add company* dialog |
| `app/static/scripts/overview.js` | 83 | Dashboard Overview: tiles, needs-attention list, rules bar, segment cards |
| `app/static/scripts/masterdata.js` | 98 | Master Data cards: company profile with save-on-change; related standing records |
| `app/static/scripts/directors.js` | 117 | Directors & KMP register and the add-to-board dialog |
| `app/static/scripts/meetings.js` | 109 | Meetings book cards, meeting detail, attendance by roster, document upload |
| `app/static/scripts/calc.js` | 77 | Calculated Values: the 13 views as tables with CSV export |
| `app/static/scripts/rules.js` | 83 | Applicability Rules with evaluations and the two computed checks |
| `app/static/scripts/activities.js` | 277 | Activities & Tasks: templates, start dialog, task workflow, confirmations, evidence download |
| `app/static/scripts/documents.js` | 79 | Documents & Evidence: drag-and-drop upload, list, link to a record or task |
| `app/static/scripts/trail.js` | 81 | Activity Log from `/api/audit` with filters and CSV export |

### Database

| File | Lines | Purpose |
|---|---|---|
| `db/migrations/001_extensions_schemas_roles.sql` | 75 | Extensions (vector, btree_gist, pgcrypto), schemas, roles |

| File | Lines | Purpose |
|---|---|---|
| `db/migrations/002_audit_and_common_functions.sql` | 174 | Audit table and trigger, common functions (touch, block delete, validate lookup) |
| `db/migrations/003_core_reference_and_rules.sql` | 104 | Reference data: lookups, document types, rule definitions |
| `db/migrations/004_core_company_people.sql` | 374 | Company, directors, directorships, designations, KYC |
| `db/migrations/005_core_capital_members.sql` | 255 | Share capital, shareholders, holdings, allotments, transfers, foreign investment |
| `db/migrations/006_core_governance.sql` | 212 | Meetings, attendance, committees, resolutions, ballots |
| `db/migrations/007_core_financial_legal_filings.sql` | 318 | Filings, loans, remuneration, penalties, Section 185/186/188, ODI, ECB |
| `db/migrations/008_core_documents_evidence_users.sql` | 164 | Documents, evidence links, chunks, users, roles, access |
| `db/migrations/009_activity_and_ops.sql` | 423 | Activity templates, steps, dependencies, bindings, evidence rules; instances, tasks, confirmations; `instantiate_activity` |
| `db/migrations/010_views.sql` | 355 | 13 calculated views |
| `db/migrations/011_triggers_rls_grants.sql` | 250 | Audit and no-delete triggers, row-level-security policies, grants |
| `db/seed/01_lookups_rules_doctypes.sql` | 622 | Standard Lists, applicability rules, document types |
| `db/seed/02_users_companies_people.sql` | 463 | Users, five companies, people |
| `db/seed/03_capital_governance_filings.sql` | 363 | Capital, governance, filings |
| `db/seed/04_activity_sample_and_instance.sql` | 570 | Sample activity template and a running instance |
| `db/seed/05_required_seed_completion.sql` | 128 | Completion of required seed data |
| `demo/demo_queries.sql` | 262 | SQL used when demonstrating the database |

### Command-line scripts

| File | Lines | Purpose |
|---|---|---|
| `scripts/setup_db.py` | 268 | Create database/roles/extensions; apply migrations and seeds ( `--seed` , `--reset` ) |
| `scripts/run_server.py` | 41 | Start the web application with uvicorn ( `--host --port --workers --reload` ) |
| `scripts/manage_users.py` | 88 | `list` , `set-password` , `init-demo` for the credentials file |
| `scripts/run_demo.py` | 372 | 10-scene Phase 1 database demonstration |
| `scripts/check_traceability.py` | 67 | Exit code 0 when every spreadsheet row is mapped in FIELD_TRACEABILITY.md |
| `scripts/generate_traceability.py` | 525 | Regenerates `docs/FIELD_TRACEABILITY.md` from the spreadsheet |

### Tests

| File | Lines | Purpose |
|---|---|---|
| `tests/conftest.py` | 56 | Fixtures for the Phase 1 database tests |
| `tests/test_activity_ops.py` | 130 | Activity engine at database level (4 tests) |
| `tests/test_lookups_and_constraints.py` | 73 | Lookup and constraint rules (3 tests) |
| `tests/test_no_delete_and_audit.py` | 129 | No deletes; audit trail (5 tests) |
| `tests/test_overlap_constraints.py` | 121 | Overlapping periods rejected (4 tests) |
| `tests/test_rls_isolation.py` | 74 | Row-level security (2 tests) |
| `tests/test_traceability.py` | 19 | Requirement traceability complete (1 test) |
| `tests/api/__init__.py` | 0 | Package marker |
| `tests/api/conftest.py` | 168 | Builds `bmp_api_test` , starts the real app, signs in three users (fixtures) |
| `tests/api/payloads.py` | 269 | `Factory` that builds a valid payload for every writable field of every table |

| File | Lines | Purpose |
|---|---|---|
| `tests/api/test_auth.py` | 89 | Sign-in, CSRF, lock-out, password change (11 tests) |
| `tests/api/test_isolation.py` | 166 | Company isolation for every table, view, export (142 tests) |
| `tests/api/test_meta_coverage.py` | 95 | Every table/column/view reachable from the UI (9 tests) |
| `tests/api/test_roundtrip_all_fields.py` | 167 | Every writable field stores exactly; stale edits; archive/restore (67 tests) |
| `tests/api/test_ui_assets.py` | 87 | Page, headers, CSS byte-identity, JavaScript syntax and ids (24 tests) |
| `tests/api/test_workflows.py` | 490 | Activities, meetings, register, documents, exports, rules, admin (35 tests) |

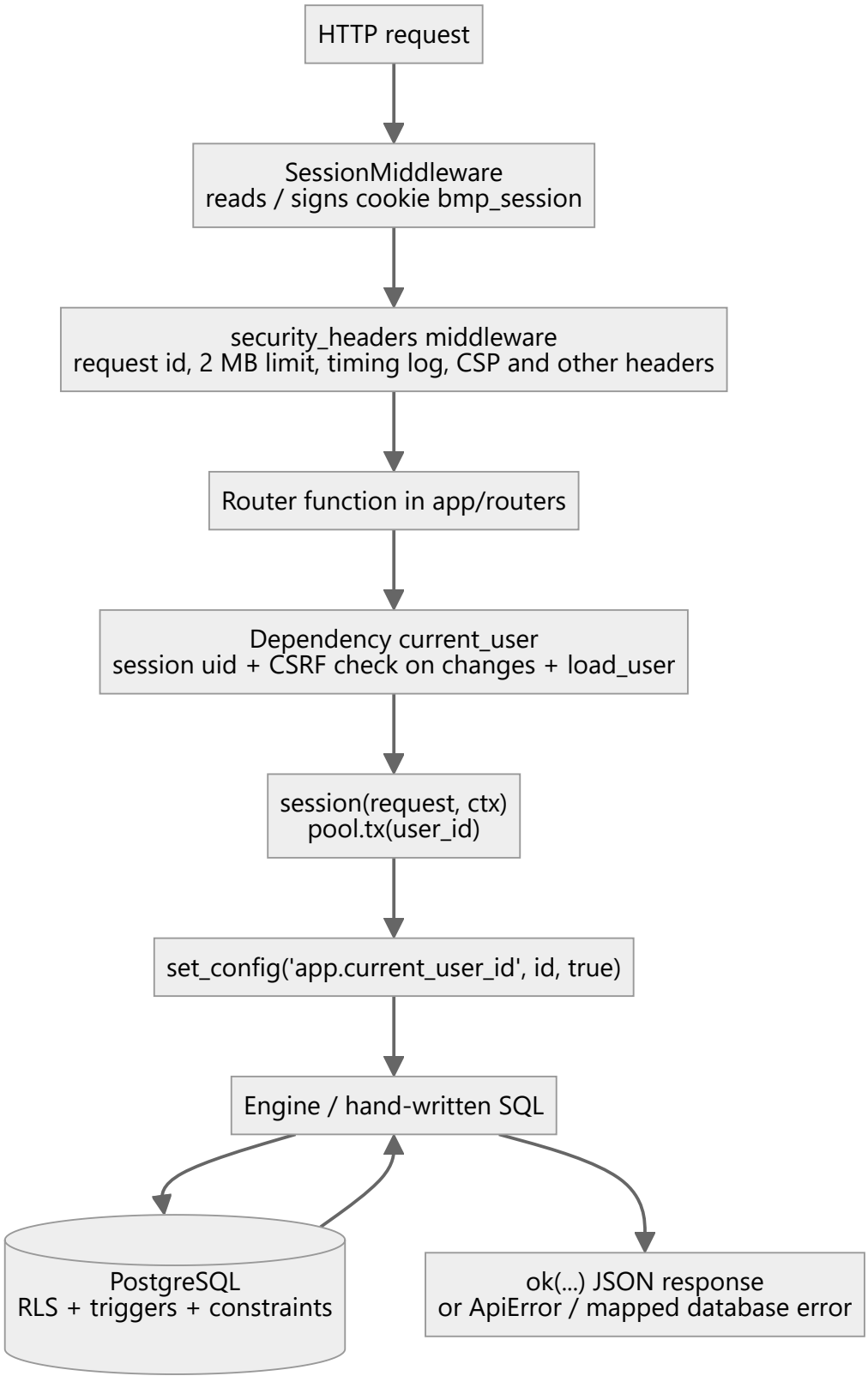
### Documentation and configuration

| File | Lines | Purpose |
|---|---|---|
| `README.md` | 27 | Quick start |
| `requirements.txt` | 11 | Python packages (pip install -r) |
| `.env.example` | 43 | Settings template (copy to `.env`) |
| `docs/ERD.md` | 816 | Entity-relationship diagrams |
| `docs/DATA_DICTIONARY.md` | 1508 | Every table and column described |
| `docs/FIELD_TRACEABILITY.md` | 308 | Spreadsheet row → database object map |
| `docs/postman/BMP_Compliance_Platform.postman_collection.json` | 149 KB | Postman collection (114 requests, 216 checks) |
| `docs/postman/BMP_Compliance_Platform.postman_environment.json` | 1 KB | Postman environment |
| `docs/postman/sample_notice.pdf` | 0 KB | Small file for the upload test |

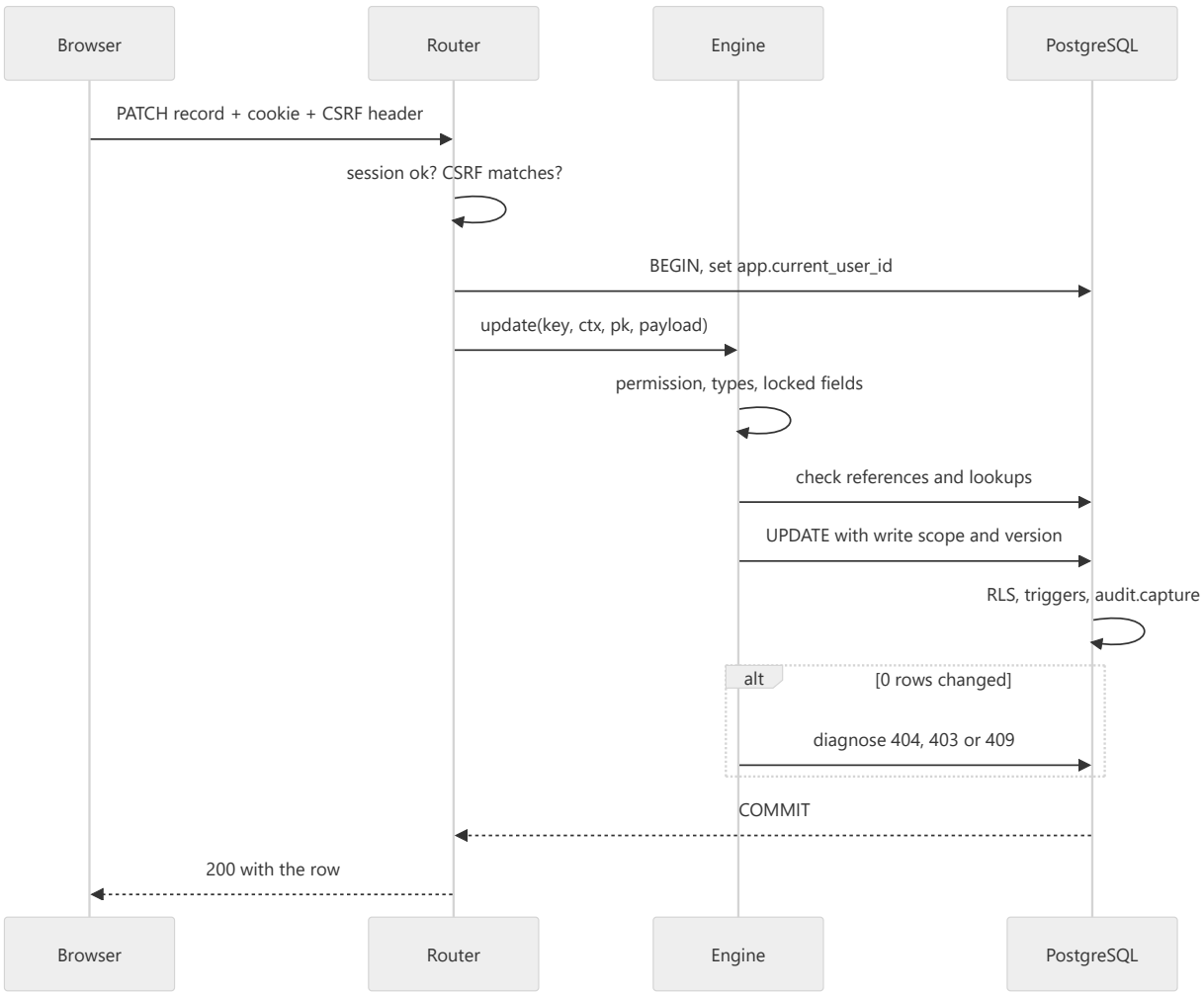
# PART 4 — The back end

## 4. Back-end design

### 4.1 Architecture and the life of a request



The same sequence for a change (here: editing a record), showing where each safeguard fires:




## 4.2 Start-up (`app/main.py`)

`create_app(settings)` builds the FastAPI object. Its **lifespan** hook runs once when the server starts:

1. Opens the **app pool** (`DATABASE_URL_APP`, 1–10 connections) and, if configured, the **privileged pool** (`DATABASE_URL_PRIVILEGED`, falling back to `DATABASE_URL`; 0–3 connections).
2. Reads the database catalog once (`schema_meta.load_catalog`) and keeps it in `app.state.catalog`.
3. Creates `app.state.engine` (the generic record engine), `app.state.users` (credentials file), `app.state.throttle` (login lock-out).
4. Creates the storage folder.

Then: session middleware, the security-headers middleware, five error handlers (always a JSON `{"error": {...}}` body), the ten routers under the `/api` prefix, `/static`, the page route `/` and `/api/health`. `uvicorn app.main:app` works because `app.main` creates the application lazily on first access, so importing the module (for example in tests) needs no environment.

| Middleware / handler | What it does |
|---|---|
| `SessionMiddleware` | Signed cookie `bmp_session`; `SameSite=Lax`; `Secure` when `COOKIE_SECURE`; lifetime `SESSION_MAX_AGE_SECONDS` |
| `security_headers` | Adds `X-Request-ID`; rejects request bodies over 2 MB (except `/documents/upload`); logs `id method path status ms user`; sets `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Content-Security-Policy`, `Cache-Control`; adds HSTS in production |

| Middleware / handler | What it does |
|---|---|
| `ApiError` handler | Turns the project's own error into `{error:{code,message,fields}}` |
| `RequestValidationError` handler | 422 with per-field messages (bad JSON body) |
| `StarletteHTTPException` handler | 404/405 etc. as `http_<status>` codes |
| `psycopg.Error` handler | Database refusal → friendly message via `errors.map_db_error` |
| Catch-all handler | 500 `internal`, details only in the server log |

### 4.3 Settings (`app/config.py` and `.env`)

Settings are read **once** at start into a frozen `Settings` object (`Settings.load()`); `.env` is loaded with python-dotenv, and real environment variables override `.env`. Nothing secret has a default; the only generated value is a temporary session key in development.

| `.env` key | Default | Setting | Effect |
|---|---|---|---|
| `DATABASE_URL` | required for set-up | `db_privileged` | Login `bmp_owner`: migrations, plus the web app's audit reader and company onboarding |
| `DATABASE_URL_APP` | **required** | `db_app` | Login `bmp_app`: every normal request (no delete, subject to row-level security) |
| `DATABASE_URL_PRIVILEGED` | = `DATABASE_URL` | `db_privileged` | Optional separate privileged login |
| `DATABASE_URL_ADMIN` | — | (not in `Settings`) | PostgreSQL administrator; used only by `setup_db.py` and the tests |
| `APP_ENV` | `development` | `env` | `production` demands a secret key, forces secure cookies + HSTS, hides `/api/docs` and `/api/openapi.json` |
| `APP_SECRET_KEY` | temporary random key (development only) | `secret_key` | Signs the session cookie; ≥ 32 characters in production |
| `SESSION_MAX_AGE_SECONDS` | 28800 (8 h) | `session_max_age` | Sign-in lifetime |
| `COOKIE_SECURE` | false (true in production) | `cookie_secure` | Cookie only over HTTPS |
| `STORAGE_DIR` | `storage` | `storage_dir` | Where uploads are stored (relative paths are inside the project) |
| `MAX_UPLOAD_MB` | 25 | `max_upload_bytes` | Upload size cap |
| `USERS_FILE` | `instance/users.json` | `users_file` | Credentials file |
| `ALLOW_PORTAL_SECRET_STORAGE` | false | `allow_portal_secret` | Allows writing the encrypted *portal secret* column |
| `ALLOW_AADHAAR_STORAGE` | false | `allow_aadhaar` | Allows writing the encrypted *Aadhaar* column |
| `APP_ENCRYPTION_KEY` | — | `encryption_key` | Required if either flag above is on (used by `pgp_sym_encrypt`) |
| `ENFORCE_TASK_CONFIRMATION` | true | `enforce_task_confirmation` | Steps marked "requires confirmation" cannot complete without one |
| `DB_POOL_MIN` / `DB_POOL_MAX` | 1 / 10 | `pool_min` / `pool_max` | App pool size |

| .env key | Default | Setting | Effect |
|---|---|---|---|
| LOGIN_MAX_FAILURES / LOGIN_WINDOW_SECONDS | 5 / 600 | login_max_failures, login_window_seconds | Lock-out rule |

Start-up refuses (with a clear message) when: APP_ENV is not development / production; DATABASE_URL_APP is missing; production has no APP_SECRET_KEY (or fewer than 32 characters); or an ALLOW_*_STORAGE flag is on without APP_ENCRYPTION_KEY.

## 4.4 Database integration

The application never defines tables. It connects to the approved PostgreSQL database and **respects its rules**:

| Concern | How it is integrated |
|---|---|
| **Two logins** | bmp_app (cannot delete or truncate; subject to row-level security) serves normal requests. bmp_owner is used in exactly three places: reading the audit log (/api/audit, record history) and onboarding a company (POST /api/companies) — things the application login is deliberately forbidden to do |
| **Connection pools** | db.Database wraps psycopg_pool.ConnectionPool (dict rows, 10 s timeout). tx(user_id, readonly) is the only way routers reach the database |
| **Who is acting** | Each request runs in one transaction that starts with SELECT set_config('app.current_user_id', <id>, true). The value is transaction-local, so it cannot leak to the next request that reuses the pooled connection |
| **Audit** | The database trigger audit.capture reads app.current_user_id and writes who / when / old row / new row to audit.log. A write without an actor is rejected. The log itself cannot be updated, deleted or truncated |
| **Row-level security** | Policies on the 32 tables that carry company_id filter every read and write by the same setting |
| **No deletes** | audit.block_delete / block_truncate triggers exist on every table except two pure link tables (activity.step_dependency, core.role_permission), and the audit log has its own block. The API offers *archive* (is_archived, archived_at, archived_by) and *restore* only |
| **Lookups** | audit.validate_lookup ensures a lookup column holds a value of the right Standard List; the API also pre-checks it to give a friendly message |
| **Overlaps** | Exclusion constraints reject overlapping periods (for example two board stints of one person at one company) → HTTP 409 overlap |
| **Optimistic lock** | touch_updated_at maintains updated_at; the API compares the version the user last saw (409 stale) |
| **Database functions called** | ops.instantiate_activity(code, company, params, user) (start an activity: creates the instance and one task per step with computed due dates), activity.get_template_json(code, version) (a template as one JSON document), the triggers activity.check_step_dependency and activity.check_write_on_calculated |
| **Views** | 13 calculated views (core.v_*). PostgreSQL views run with their owner's rights and so **bypass row-level security**; the API therefore re-applies company scope itself (Engine.view_rows) |
| **Encrypted columns** | bytea columns (Aadhaar, portal secret) are written with pgp_sym_encrypt(value, APP_ENCRYPTION_KEY) **only** when the policy flag allows it; ciphertext is never returned (the API sends just *is it set?*) |
| **Catalog introspection** | schema_meta.load_catalog queries pg_catalog for columns, types, primary/unique/foreign keys, CHECK ... = ANY(ARRAY[...]) enums, the financial-year pattern and validate_lookup trigger bindings, so API and forms follow the schema automatically |
| **Seed IDs the demo relies on** | Company 1 = Nexatech, 2 = BrightMed Pharma, … (see db/seed/02_*); the Postman environment uses these |

The privileged connection is only opened when a privileged URL is configured; without it the three privileged actions return 501 not_configured rather than failing obscurely.

## 4.5 API files

**Router modules** (`app/routers/`, all mounted under `/api`):

| File | Lines | Endpoints | Responsibility |
|---|---|---|---|
| `auth.py` | 101 | 5 | Sign in/out, session info, password change, admin password set; throttle and CSRF token creation |
| `meta.py` | 159 | 2 | The self-describing metadata and dropdown options; label humanising (`cin` → *CIN*) |
| `entities.py` | 189 | 8 | Generic list/get/create/patch/archive/restore/history/CSV for any record type; locked workflow fields; `entity_type` / `entity_id` reference checks |
| `companies.py` | 238 | 6 | Company list, onboarding (privileged), financial years, the overview calculations, JSON and Excel export |
| `people.py` | 83 | 2 | Directors register query; add a person to a board in one transaction (reuse by DIN) |
| `meetings.py` | 113 | 3 | Meetings book with agenda/attendance; one meeting with roster and documents; save attendance |
| `views.py` | 49 | 2 | Calculated views per user scope; applicability rules with evaluations and two computed checks |
| `activities.py` | 315 | 10 | Templates, start an activity, task/instance status workflow, confirmations |
| `documents.py` | 111 | 2 | Evidence upload (hash, version, store, link) and checked download |
| `audit.py` | 77 | 1 | The Activity Log, read on the privileged connection and narrowed to the user's companies |

**Support modules** (`app/`): `main.py` (163 lines), `config.py` (117), `db.py` (51), `deps.py` (63), `access.py` (166), `schema_meta.py` (225), `registry.py` (433), `crud.py` (473), `coerce.py` (106), `errors.py` (107), `security.py` (114), `responses.py` (32). See section 7 for the important ones in depth.

## 4.6 API conventions

**Successful responses** are plain JSON objects whose key names the content (`{"row": {...}}`, `{"rows": [...], "total": 8}`, `{"companies": [...]}`). Numbers that must stay exact (money) and dates are sent as **strings**. Creation returns **201**.

**Errors** always have the same shape:

```
{ "error": { "code": "validation",
             "message": "Please correct the highlighted fields.",
             "fields": { "name": "Required", "valid_from": "Required" } } }
```

| HTTP | `error.code` | Meaning |
|---|---|---|
| 401 | `unauthenticated` | Not signed in, session expired, or the account is no longer active |
| 401 | `bad_credentials` | Wrong e-mail or password at sign-in |
| 403 | `bad_credentials` | Wrong *current* password when changing a password |
| 403 | `csrf` | Missing or wrong `X-CSRF-Token` on a change |
| 403 | `forbidden` | Read-only access, or administrator-only action |
| 403 | `policy` | Writing an encrypted field that is switched off by policy |
| 404 | `not_found` | Does not exist **or you may not see it** (existence is not revealed) |
| 404 | `file_missing` | The document row exists but the file is not in storage |
| 405 | `use_workflow` | Create company / activity / task through their dedicated action |
| 405 | `http_405` | Method not offered (there is no DELETE anywhere) |
| 409 | `stale` | Someone changed the record since you loaded it |
| 409 | `blocked` | An earlier step of the activity is not done |

| HTTP | error.code | Meaning |
|---|---|---|
| 409 | `needs_confirmation` | The step requires a named person's confirmation first |
| 409 | `duplicate` / `overlap` | Unique violation / overlapping period |
| 409 | `integrity` | The stored file no longer matches its recorded SHA-256 |
| 409 | `conflict` | Other workflow conflicts (cancelled activity, decided confirmation, …) |
| 413 | `too_large` | Request body over 2 MB (uploads allow `MAX_UPLOAD_MB`) |
| 422 | `validation` | A field is missing or wrong; `fields` names each one |
| 429 | `too_many_attempts` | Sign-in lock-out |
| 501 | `not_configured` | A privileged action needs `DATABASE_URL_PRIVILEGED` |
| 503 | `unavailable` | Database not reachable |
| 500 | `internal` | Unexpected; logged with a stack trace, never shown |

**Other conventions**

- **Record types** are addressed by table name (`/api/entities/meeting`). 65 are visible to an administrator; a read-only client sees fewer (administration tables answer 404).
- **Primary keys** are strings in the URL; a composite key is joined with `~`.
- **Lists** accept `company_id`, `fy` (financial year such as `2024-25`), `q` (text search), `archived=true`, `sort`, `desc`, `page`, `size` (≤ 5000) and **column filters** `f_<column>=value` (for example `f_status=held`). They return `{entity, total, page, size, rows}`; every row also has `_pk` and `_labels` (readable text for each foreign key).
- **Edits** carry `updated_at` (the version the person saw). A `PATCH` with a stale value fails with 409 `stale`.
- **System columns** (`created_at`, `updated_at`, `is_archived`, …), identity and generated columns are ignored if sent.
- **Financial year** values must look like `2024-25`.

## 4.7 Complete endpoint reference

43 routes: 41 API endpoints, the health probe and the dashboard page. *Access:* **Public** = no sign-in; **User** = any active signed-in user, further limited to the companies they may see; **Admin** = administrators only. All changes (POST/PUT/PATCH) additionally need the `X-CSRF-Token` header.

### Authentication (`routers/auth.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **POST** | `/api/auth/login` | Public | body: `email`, `password` | Sign in. Returns `user` and `csrf`; sets the session cookie. 401 `bad_credentials`; 429 after repeated failures. |
| **POST** | `/api/auth/logout` | Public | — | End the session (CSRF header required when a session exists). |
| **GET** | `/api/auth/me` | User | — | Current user, company access and CSRF token; used when the page reloads. |
| **POST** | `/api/auth/password` | User | body: `current_password`, `new_password` | Change own password (at least 8 characters). |
| **POST** | `/api/admin/users/{user_id}/password` | Admin | body: `new_password` | Administrator sets another user's sign-in password. |

### Metadata and options (`routers/meta.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/meta` | User | — | Self-description: `entities` (columns, kinds, choices, lookups, write rights), `views`, `nav`, `lookups`, `user`, `policy`. |
| **GET** | `/api/options/{entity}` | User | `q`, `company_id`, `ids` (comma list), `limit` (≤ 200) | Dropdown options `[{id, label}]` for any record type, limited to what the user may see. |

### Generic records (`routers/entities.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/entities/{key}` | User | `company_id`, `fy`, `q`, `archived`, `sort`, `desc`, `page`, `size`, `f_<column>` | List records: `{total, page, size, rows}`; each row has `_pk` and `_labels`. |
| **GET** | `/api/entities/{key}/export.csv` | User | `company_id`, `fy`, `q`, `f_<column>` | The list as a CSV file (UTF-8 with BOM; formula-injection safe). |
| **GET** | `/api/entities/{key}/{pk}` | User | — | One record (composite keys joined by `~`). |
| **POST** | `/api/entities/{key}` | User | body: the record's fields | Create a record (201). Company, activity instance and task use their own actions (405 `use_workflow`). |
| **PATCH** | `/api/entities/{key}/{pk}` | User | body: changed fields + `updated_at` | Edit with optimistic lock (409 `stale`); workflow-controlled fields are refused. |
| **POST** | `/api/entities/{key}/{pk}/archive` | User | — | Hide the record. Nothing is deleted. |
| **POST** | `/api/entities/{key}/{pk}/restore` | User | — | Un-hide an archived record. |
| **GET** | `/api/entities/{key}/{pk}/history` | User | — | Audit entries for the record: operation, time, person, each changed field *from* → *to* (latest 200). |

### Companies (`routers/companies.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/companies` | User | — | Companies the user may see, with director and overdue-filing counts and the user's access level. |
| **POST** | `/api/companies` | Admin | body: company fields | Onboard a company (administrator). Runs on the privileged connection and gives the creator admin access (201). |
| **GET** | `/api/companies/{company_id}/years` | User | — | Financial years that have data, and the current year. |
| **GET** | `/api/companies/{company_id}/overview` | User | `fy` | Dashboard: tiles, needs-attention list, rules summary, record counts per group. |
| **GET** | `/api/companies/{company_id}/export.json` | User | — | Whole-company export, one section per record type. |
| **GET** | `/api/companies/{company_id}/export.xlsx` | User | — | The same as an Excel workbook, one sheet per record type. |

### Directors register (`routers/people.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/companies/{company_id}/directors-register` | User | `include_ceased` (default true) | Every board stint with person, designation, age and a current flag. |
| **POST** | `/api/companies/{company_id}/directors-register` | User | body: `director` {...} or `director_id`, `directorship` {`start_date`, ...}, `designation_id`, `designation_start` | Add a person to the board in one transaction; reuses an existing person by DIN (201). Overlapping stint → 409. |

### Meetings (`routers/meetings.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/companies/{company_id}/meetings-book` | User | `fy` | Meetings with agenda items and attendance. |
| **GET** | `/api/meetings/{meeting_id}/detail` | User | — | One meeting with the **roster** (directors on the board that day), attendance, agenda and documents. |
| **PUT** | `/api/meetings/{meeting_id}/attendance` | User | body: `rows` [{ `directorship_id`, `attended`, `mode` }] | Create or update one attendance row per director; never deletes. |

### Calculated values and rules (`routers/views.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/calc/{view}` | User | `company_id`, `limit` | Rows of one of the 13 calculated views, limited to the user's companies. |
| **GET** | `/api/companies/{company_id}/rules` | User | `fy` | Applicability rules with the latest evaluation, plus two rules computed from the board's records. |

### Activities and tasks (`routers/activities.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/activities/templates` | User | — | Active/draft templates with step counts. |
| **GET** | `/api/activities/templates/{template_id}` | User | — | A template with steps, dependencies, data bindings, evidence requirements and deadlines. |
| **GET** | `/api/activities/templates/{code}/json` | User | `version` | The raw `activity.get_template_json` document (the shape a future MCP tool would read). |
| **POST** | `/api/activities/instances` | User | body: `template_code`, `company_id`, `params` | Start an activity; checks `params` against the template's schema; creates the tasks with due dates (201). |
| **GET** | `/api/companies/{company_id}/activities` | User | — | A company's activities with task, done and overdue counts. |
| **GET** | `/api/activities/instances/{instance_id}` | User | — | An activity with each task: blockers, evidence, links, confirmations, overdue flag. |

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **POST** | `/api/activities/tasks/{task_id}/status` | User | body: `status`, `notes` | Move a task under the workflow rules (order, confirmation); returns `warnings` for missing evidence. |
| **POST** | `/api/activities/instances/{instance_id}/status` | User | body: `status` = `active` or `cancelled` | Cancel or reopen an activity. |
| **POST** | `/api/activities/tasks/{task_id}/confirmations` | User | body: `{}` | Request a named person's confirmation (201); one pending at a time. |
| **POST** | `/api/confirmations/{confirmation_id}/{decision}` | User | `decision` = `confirm` or `reject` | Decide a confirmation; someone other than the requester (an administrator may decide their own). |

### Documents (`routers/documents.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **POST** | `/api/documents/upload` | User | multipart: `company_id`, `file`, `document_type_id`, `entity_type`, `entity_id`, `task_instance_id`, `note` | Upload evidence: hashed, versioned, stored, optionally linked (201). |
| **GET** | `/api/documents/{document_id}/download` | User | — | Download as an attachment after re-checking the SHA-256. |

### Audit (`routers/audit.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/audit` | User | `company_id`, `table`, `q`, `limit`, `offset` | The Activity Log, narrowed to the user's companies. |

### Application (`main.py`)

| Method | Path | Access | Inputs | What it does |
|---|---|---|---|---|
| **GET** | `/api/health` | Public | — | Health probe `{status, database, version}`; 503 when the database is down. |
| **GET** | `/` | Public | — | The dashboard HTML page. |

## 4.8 Security controls at a glance

| Control | Where | Detail |
|---|---|---|
| Password storage | `security.py` | scrypt ($N=2^{14}$, r=8, p=1), 16-byte salt, constant-time compare; unknown users cost the same time as wrong passwords |
| Sessions | `main.py`, `auth.py` | Signed cookie; **new session on every sign-in** (no fixation); `SameSite=Lax` |
| CSRF | `deps.current_user` | Header `X-CSRF-Token` must equal the token in the session on every non-GET request |
| Lock-out | `security.LoginThrottle` | 5 failures / 10 min per (address, e-mail) |
| Company isolation | RLS + `access.Scope` | Two layers that must agree; tests loop over every table |
| Write control | `crud.Engine`, `access.can_write_entity` | Read-only users cannot write; write modes `company`, `staff`, `admin`, `user`, `none` |
| Reference safety | `crud._check_refs` | A foreign key must point at a visible row **in the same company** |

| Control | Where | Detail |
|---|---|---|
| Headers | `main.security_headers` | CSP ( `default-src 'self'` , no inline script), `X-Frame-Options: DENY` , `nosniff` , `Referrer-Policy` , HSTS in production |
| Upload safety | `documents.py` | Extension allow-list, safe filename, size cap, SHA-256, stored outside the web root, path-traversal guard, integrity re-check on download |
| Export safety | `entities.py` , `companies.py` | CSV/Excel cells starting `= + - @` are neutralised (formula-injection guard) |
| Secrets | `crud._prepare` , `meta.py` | Encrypted columns writable only when policy flags and `APP_ENCRYPTION_KEY` are set; never returned |

# PART 5 — The front end

## 5. Front-end design

### 5.1 How the UI is delivered

The browser receives **one HTML page** ( `/` , rendered by Jinja2 from `app/templates/dashboard.html` ) that contains the sign-in card and the application shell (top bar, sidebar, content area, modal host). It loads Bootstrap and the approved `dashboard.css` from `/static` (no internet, no CDN), then `main.js` as an **ES module**. There is **no build step**: the browser loads the files as they are, so nothing needs compiling and nothing can go stale. The Content-Security-Policy forbids inline scripts, so every script is a file.

The look is the *approved dashboard*: `dashboard.css` is byte-identical to the original (a test enforces this) and `app.css` only **adds** rules for the new screens, reusing the same colour tokens.

### 5.2 Front-end files

**Page and styles**

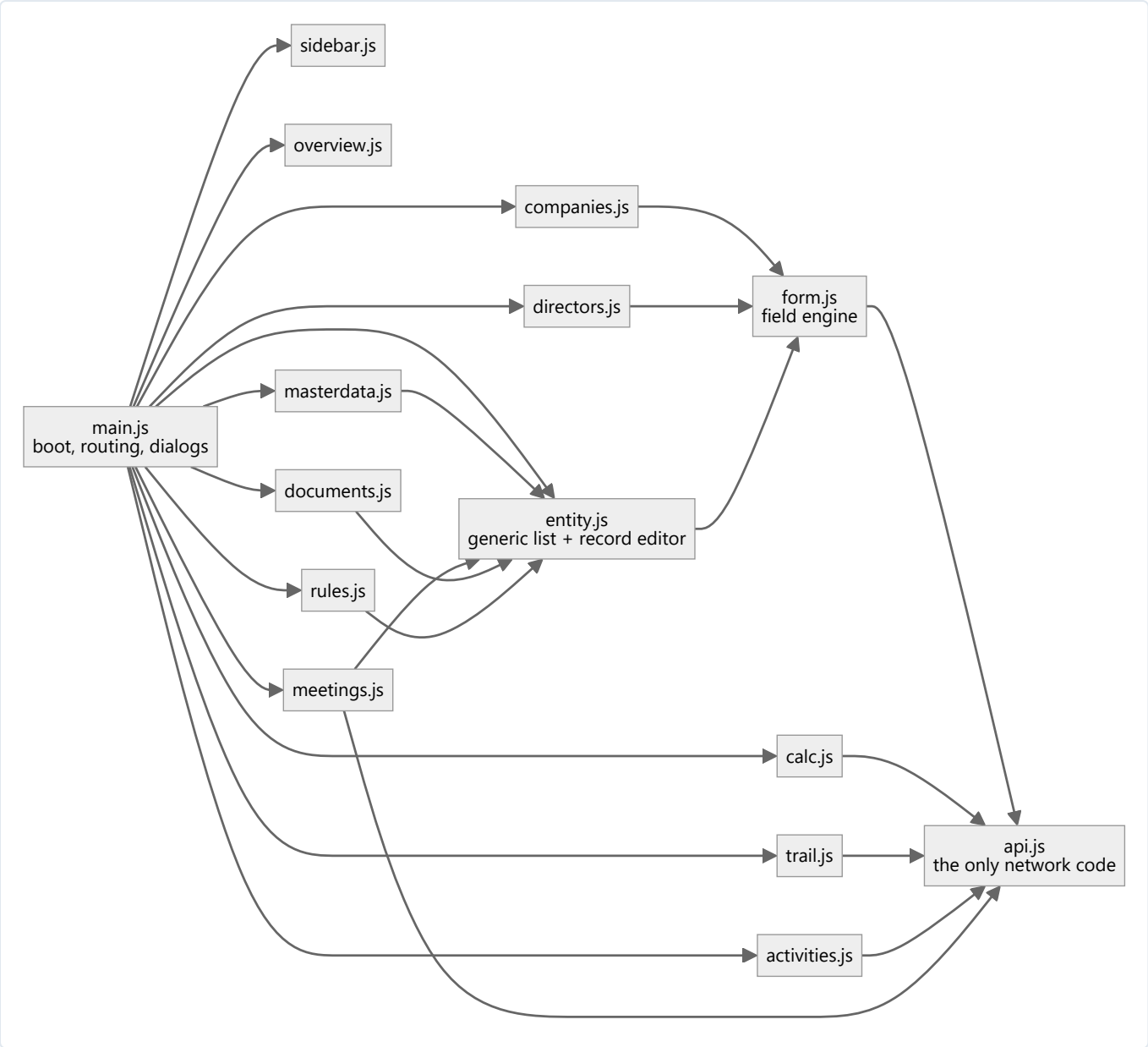
| File | Lines | Purpose |
|---|---|---|
| `app/templates/dashboard.html` | 150 | The single page: `#loginView` (sign-in), `#appShell` (header with year picker, export and user menu; sidebar accordion `#sideAcc` ; `#view` content; `#modalHost` ; toast). Loads Bootstrap and `main.js` |
| `app/static/stylesheets/dashboard.css` | 601 | Approved theme. **Never edited** |
| `app/static/stylesheets/app.css` | 115 | Additions: sign-in card, signed-in box, form fields, modals, upload drop-zone, tables |
| `app/static/vendor/bootstrap.min.css` , `bootstrap.bundle.min.js` | — | Bootstrap, served locally |

**JavaScript modules** ( `app/static/scripts/` ; *Imports* = other project modules it uses):

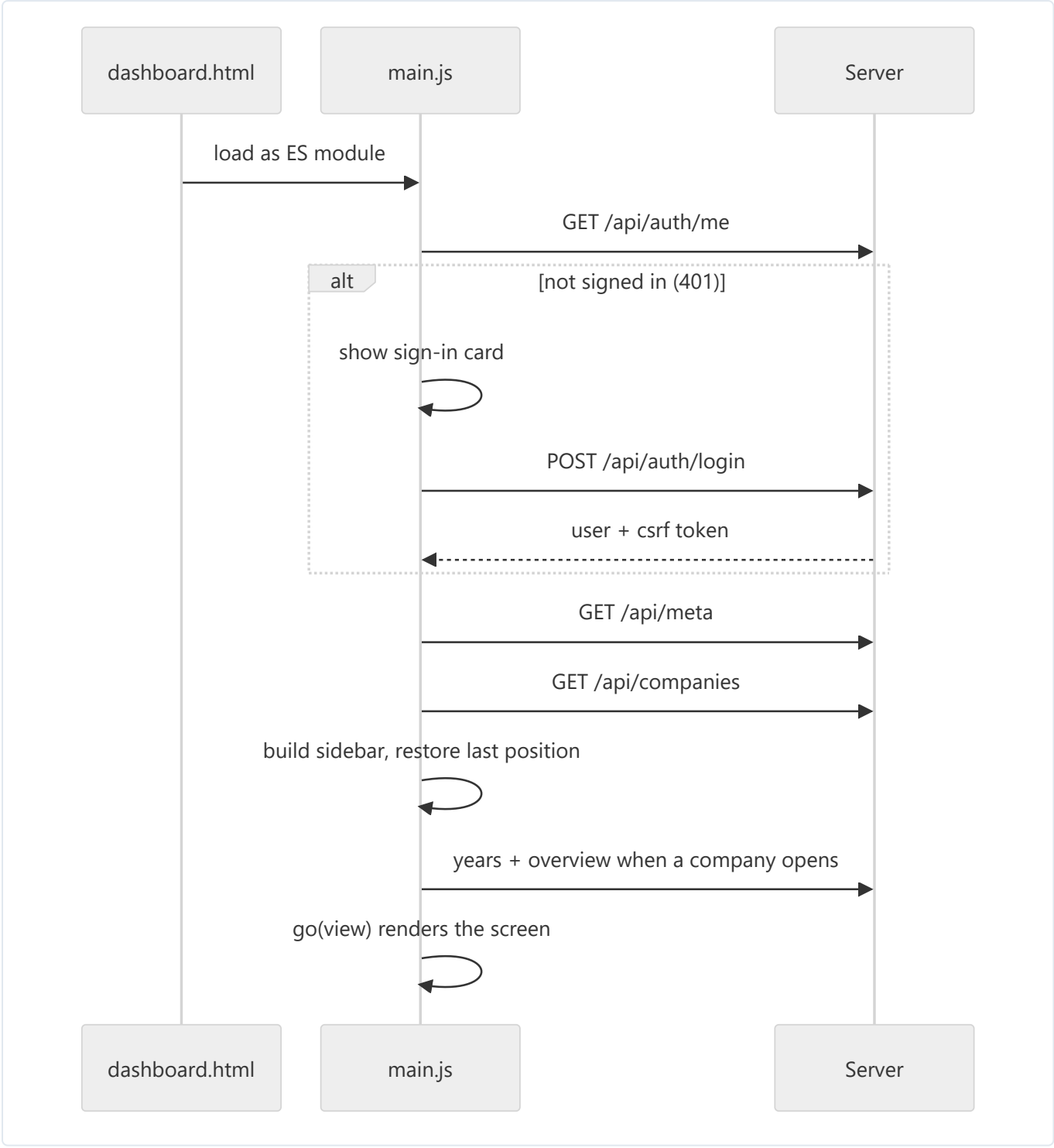
| File | Lines | Purpose | Key exports | Uses |
|---|---|---|---|---|
| `main.js` | 270 | Entry point: sign in/out, load metadata and companies, open a company, routing ( `go` ), password dialog, header buttons | `go` | `activities` , `api` , `calc` , `companies` , `directors` , `documents` , `entity` , `masterdata` , `meetings` , `overview` , `rules` , `sidebar` , `state` , `trail` , `util` |
| `api.js` | 89 | The **only** network code: `get/post/patch/put/upload/download` , `ApiError` , CSRF header, 401 event, named calls | `ApiError` , `get` , `post` , `patch` , `put` , `upload` , `download` , `login` , `logout` , `me` , `getMeta` , `getCompanies` … | `state` |
| `state.js` | 48 | The one shared `state` object plus helpers ( `company` , `entityMeta` , `canWriteCompany` , `lookup` ) | `state` , `company` , `entityMeta` , `colMeta` , `canWriteCompany` , `canWriteEntity` , `lookup` , `lookupLabel` | — |
| `util.js` | 135 | Helpers: `$` , `esc` (HTML escaping), toast, date/number formatting, CSV, drop-zone, debounce | `esc` , `toast` , `fmtDate` , `fmtDateTime` , `fmtTime` , `fmtNumber` , `toLocalInput` , `debounce` , `toCSV` , `downloadFile` , `slug` , `wireDropzone` … | — |

| File | Lines | Purpose | Key exports | Uses |
|---|---|---|---|---|
| `sidebar.js` | 79 | Builds the accordion from the server's `nav`; icons; active link; count chips | `GROUP_ICONS`, `buildSidebar`, `setActiveLink`, `updateChips` | `state`, `util` |
| `form.js` | 250 | Field engine: draws a control for any column from its metadata, collects values, shows server errors | `displayValue`, `control`, `fieldHTML`, `hydrateFk`, `collect`, `clearErrors`, `showErrors`, `sectionsFor` | `api`, `state`, `util` |
| `entity.js` | 452 | Generic table page (search, sort, page, archive toggle, CSV) and record modal (fields, children, history, archive) | `openModal`, `closeModal`, `mountList`, `openRecord`, `renderEntityPage` | `api`, `form`, `state`, `util` |
| `companies.js` | 74 | Companies landing page and the *Add company* dialog | `renderCompanies`, `openAddCompany` | `api`, `entity`, `form`, `state`, `util` |
| `overview.js` | 83 | Dashboard Overview: tiles, needs-attention list, rules bar, segment cards | `renderOverview`, `updateOverallBadge` | `sidebar`, `state`, `util` |
| `masterdata.js` | 98 | Master Data cards: company profile with save-on-change; related standing records | `render` | `api`, `entity`, `form`, `state`, `util` |
| `directors.js` | 117 | Directors & KMP register and the add-to-board dialog | `render` | `api`, `entity`, `form`, `state`, `util` |
| `meetings.js` | 109 | Meetings book cards, meeting detail, attendance by roster, document upload | `render` | `api`, `documents`, `entity`, `state`, `util` |
| `calc.js` | 77 | Calculated Values: the 13 views as tables with CSV export | `render` | `api`, `state`, `util` |
| `rules.js` | 83 | Applicability Rules with evaluations and the two computed checks | `render` | `api`, `entity`, `state`, `util` |
| `activities.js` | 277 | Activities & Tasks: templates, start dialog, task workflow, confirmations, evidence download | `render` | `api`, `documents`, `entity`, `state`, `util` |
| `documents.js` | 79 | Documents & Evidence: drag-and-drop upload, list, link to a record or task | `openUploadModal`, `render` | `api`, `entity`, `form`, `state`, `util` |
| `trail.js` | 81 | Activity Log from `/api/audit` with filters and CSV export | `render` | `api`, `state`, `util` |

## 5.3 How the modules fit together



## 5.4 Boot sequence and routing




All modules read the shared `state.js` object and use the helpers in `util.js`. `state.current` holds which view is open: a **named view** (`overview`, `masterdata`, `directors`, `meetings`, `calc`, `rules`, `activities`, `documents`, `trail`, `companies`) or `entity:<table>` for any generic table page. `go(view)` stores it, calls the right module's `render`, highlights the sidebar link and saves the position. Views that need a company ask the person to open one first. A global `bmp:unauthenticated` event (raised by `api.js` on any 401) returns the person to the sign-in card.

## 5.5 The metadata-driven field engine (`form.js`)

Nothing about a table's fields is written in JavaScript. `GET /api/meta` describes every column: `kind` (`text`, `textarea`, `int`, `decimal`, `bool`, `date`, `time`, `datetime`, `json`, `bytea`, `vector`), `required`, `choices` (from a CHECK constraint → dropdown), `lookup` (Standard List → dropdown), `fk` (another table → searchable picker through

`/api/options/<table>`), `format` (financial year), `scale` (decimals), `help` text, and for sensitive columns `secret` / `secret_enabled`. `form.fieldHTML` draws the right control, `form.collect` reads the values back and converts them, and `form.showErrors` places the server's per-field messages beside the fields. **If a column is added to the database, it appears on the form and in lists with no JavaScript change** — the round-trip tests then exercise it.

`entity.js` builds on this: a **list** (search, sort, pagination, show-archived, CSV export, "+ Add") and a **record modal** (all fields grouped in the sections defined in `registry.py`, related child records, a History tab fed by `/history`, Archive/Restore, and for the `document` and `app_user` records a *Download* and *Set password* action). Modals stack, so a related record can be opened from inside another.

## 5.6 Screens and the API calls behind them

| Screen (sidebar) | Module | Main API calls |
|---|---|---|
| All companies / Add company | `companies.js` | `GET /companies`, `POST /companies` |
| Dashboard Overview | `overview.js` (data from `main.js`) | `GET /companies/{id}/overview`, `/years` |
| Master Data | `masterdata.js` | `GET` and `PATCH /entities/company` (saves each field on change), lists of related records, `GET …/directors-register` |
| Directors & KMP Register | `directors.js` | `GET/POST /companies/{id}/directors-register`; dropdowns through `form.js` (`/options/{table}`) |
| Minutes & Meetings | `meetings.js` | `GET /companies/{id}/meetings-book`, `GET /meetings/{id}/detail`, `PUT /meetings/{id}/attendance` |
| Calculated Values | `calc.js` | `GET /calc/{view}` |
| Applicability Rules | `rules.js` | `GET /companies/{id}/rules`; *Record evaluation* opens the generic editor (`POST /entities/rule_evaluation`) |
| Activities & Tasks | `activities.js` | `/activities/templates…`, `/activities/instances…`, `/activities/tasks/{id}/status`, confirmations |
| Documents & Evidence | `documents.js` | `POST /documents/upload`, `GET /documents/{id}/download` |
| Activity Log | `trail.js` | `GET /audit` |
| Every other sidebar item | `entity.js` | `GET/POST/PATCH /entities/{key}…`, `export.csv`, `history` |
| Header: export, password | `main.js` | `GET /companies/{id}/export.xlsx\|json`, `POST /auth/password` |

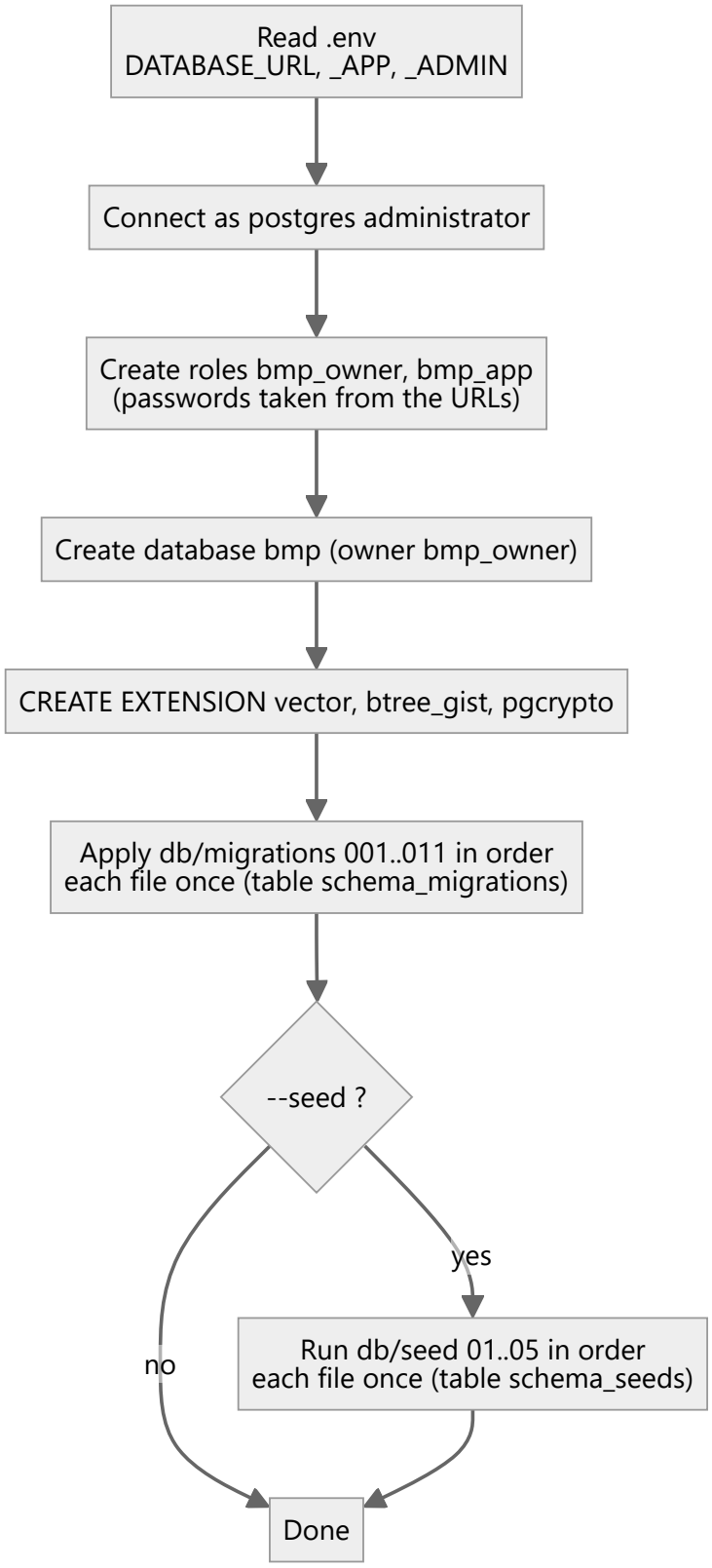
## 5.7 Extending the front end

| I want to… | Do this |
|---|---|
| Add a field to an existing table | Add a database migration. The API and forms pick it up. Optionally place it in a `sections` group in `registry.py` |
| Add a table | Migration, then add an entry to `ENTITIES` (label, list, search, label_sql, write mode) and, where it belongs in a menu, to `NAV`. If it has no `company_id`, add a `SCOPE` entry. The meta-coverage tests will tell you if anything is missing |
| Add a custom screen | Create `scripts/<name>.js` exporting `render()`, import it in `main.js`, add it to `COMPANY_VIEWS` and the `render()` switch, add a `{"view": "<name>", ...}` item in `NAV` |
| Change the look | Edit `app.css` only; leave `dashboard.css` byte-identical (a test checks) |

# PART 6 — Pipelines

## 6. Pipelines

### 6.1 Database build pipeline ( `scripts/setup_db.py` )

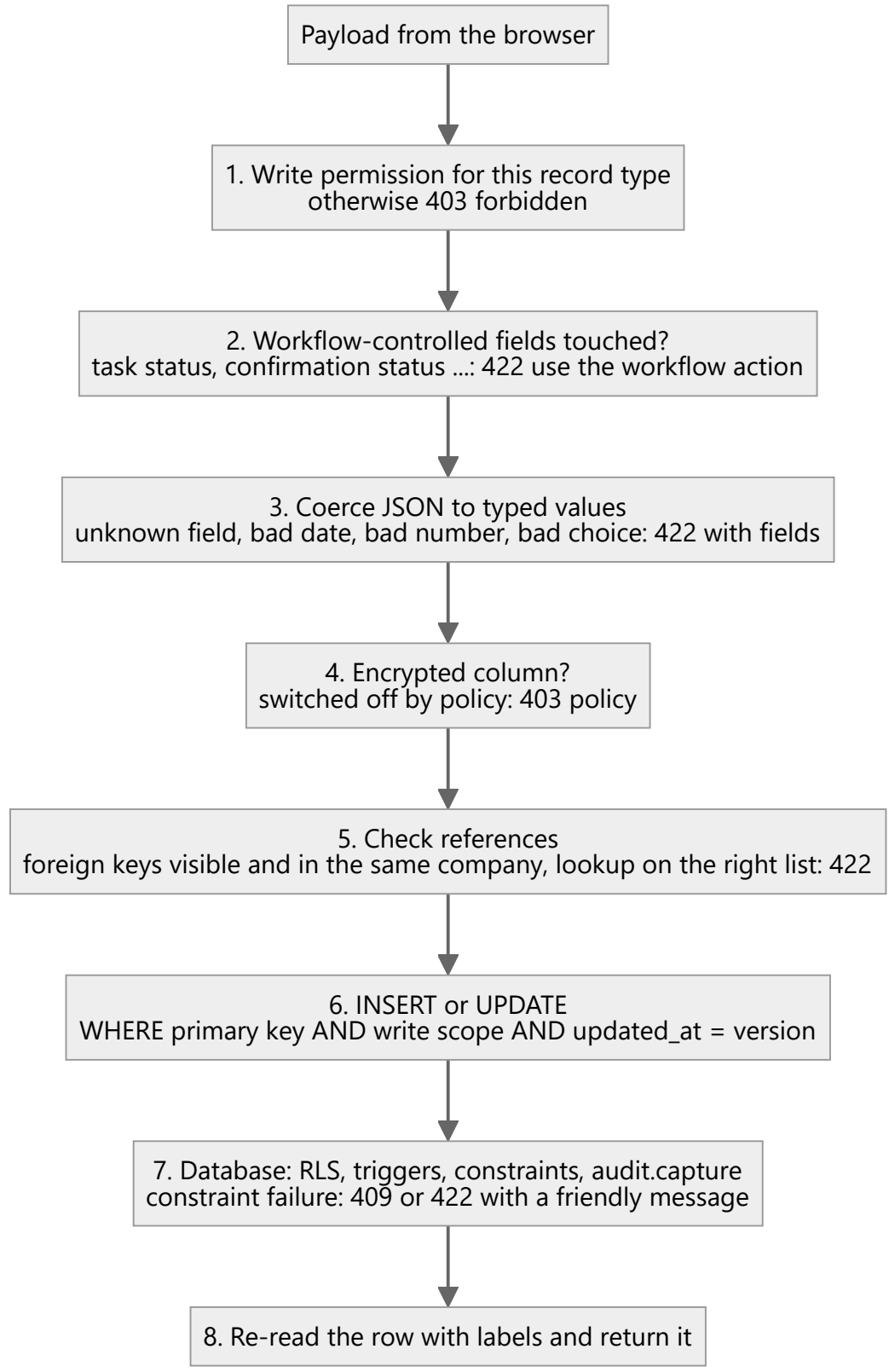


`--reset` drops the database first (development only). Migrations are recorded in `schema_migrations` and seed files in `schema_seeds`, so re-running the script applies only what is new and `--seed` is safe to repeat. To reload the seed data from scratch use `--reset --seed` (development only).

## 6.2 Request pipeline

Described in section 4.1: signed cookie → security headers → router → `current_user` (session + CSRF + active account) → transaction with `app.current_user_id` → generic engine or hand-written SQL → database policies → JSON or mapped error.

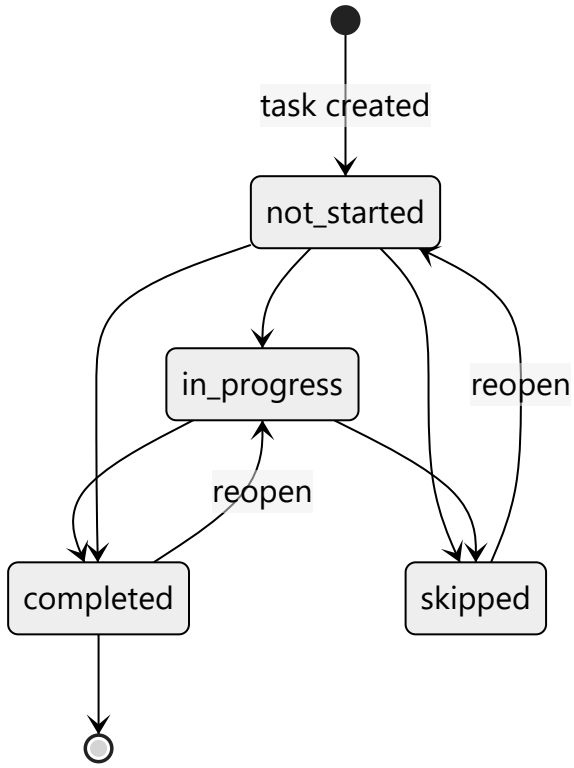
### 6.3 Write pipeline (create / edit / archive)




Archive and restore are the same pipeline with a fixed `UPDATE` of the three archive columns. **There is no delete.**

### 6.4 Activity workflow pipeline

An **activity** is a recipe (template) of ordered steps. Starting one calls `ops.instantiate_activity`, which creates the instance and one **task** per step with a **due date computed from the meeting date** (for example *notice 7 days before*, *minutes 15 and 30 days after*).

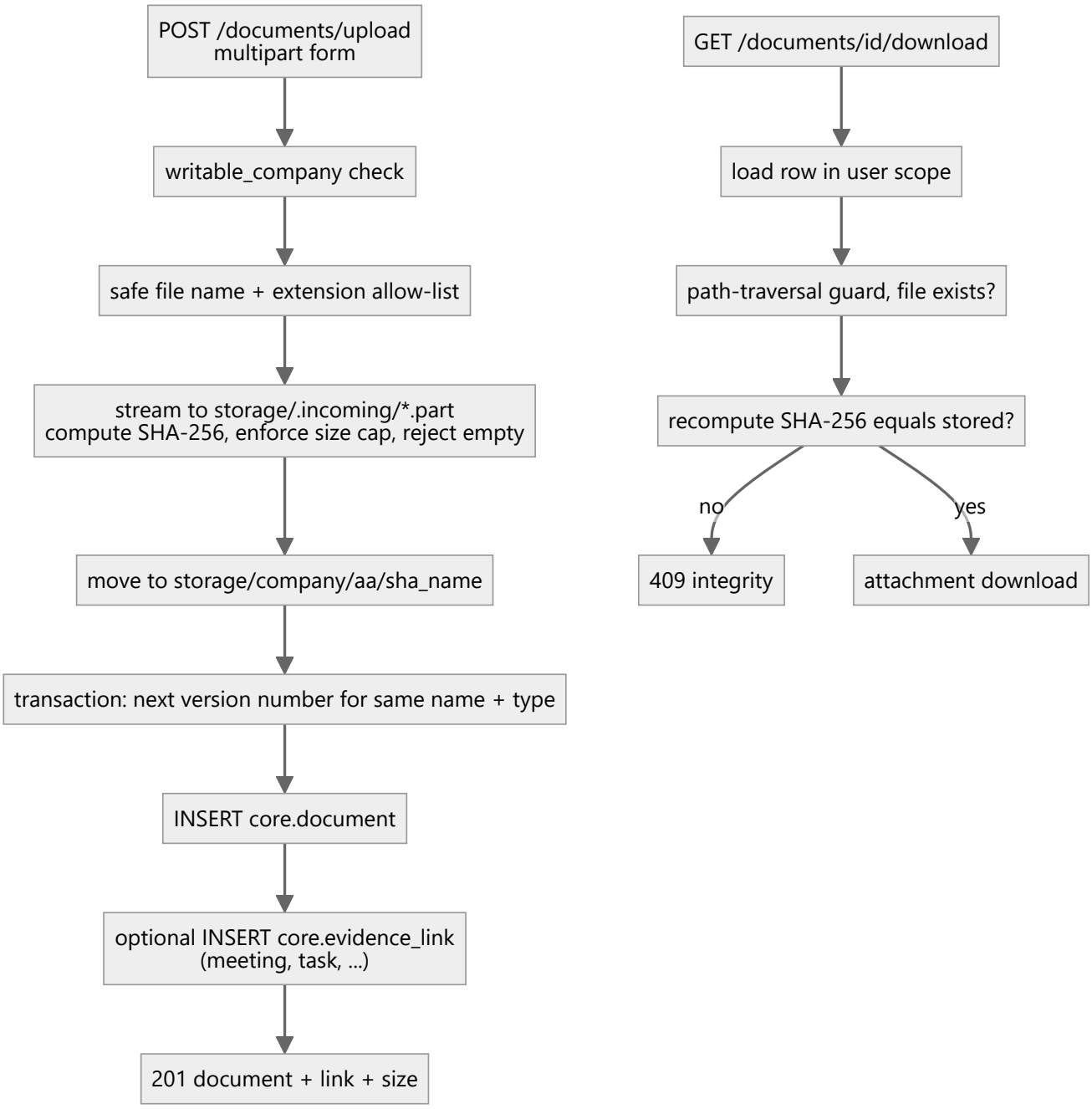



The diagram shows the normal path; the API accepts a move between any two of the four statuses, and the only gated move is **to completed**. Rules enforced by `routers/activities.py` when a task is set to **completed**:

1. **Order** — every step it depends on must be completed or skipped (otherwise 409 `blocked`).
2. **Confirmation** — if the step requires it (and `ENFORCE_TASK_CONFIRMATION` is on), a *confirmed* record must exist (otherwise 409 `needs_confirmation`). A person other than the requester must confirm; an administrator may confirm their own request.
3. **Evidence** — missing mandatory evidence does not block; it returns `warnings`.
4. `completed_by` and `completed_at` are set automatically; moving away from completed clears them.
5. **Roll-up** — when every task is completed or skipped the activity becomes `completed`; reopening a task makes it `active` again. An activity can be `cancelled` and reopened; nothing moves while cancelled.

The generic PATCH endpoint refuses to touch workflow fields (`status`, `completed_by`, …); they change only through these actions.

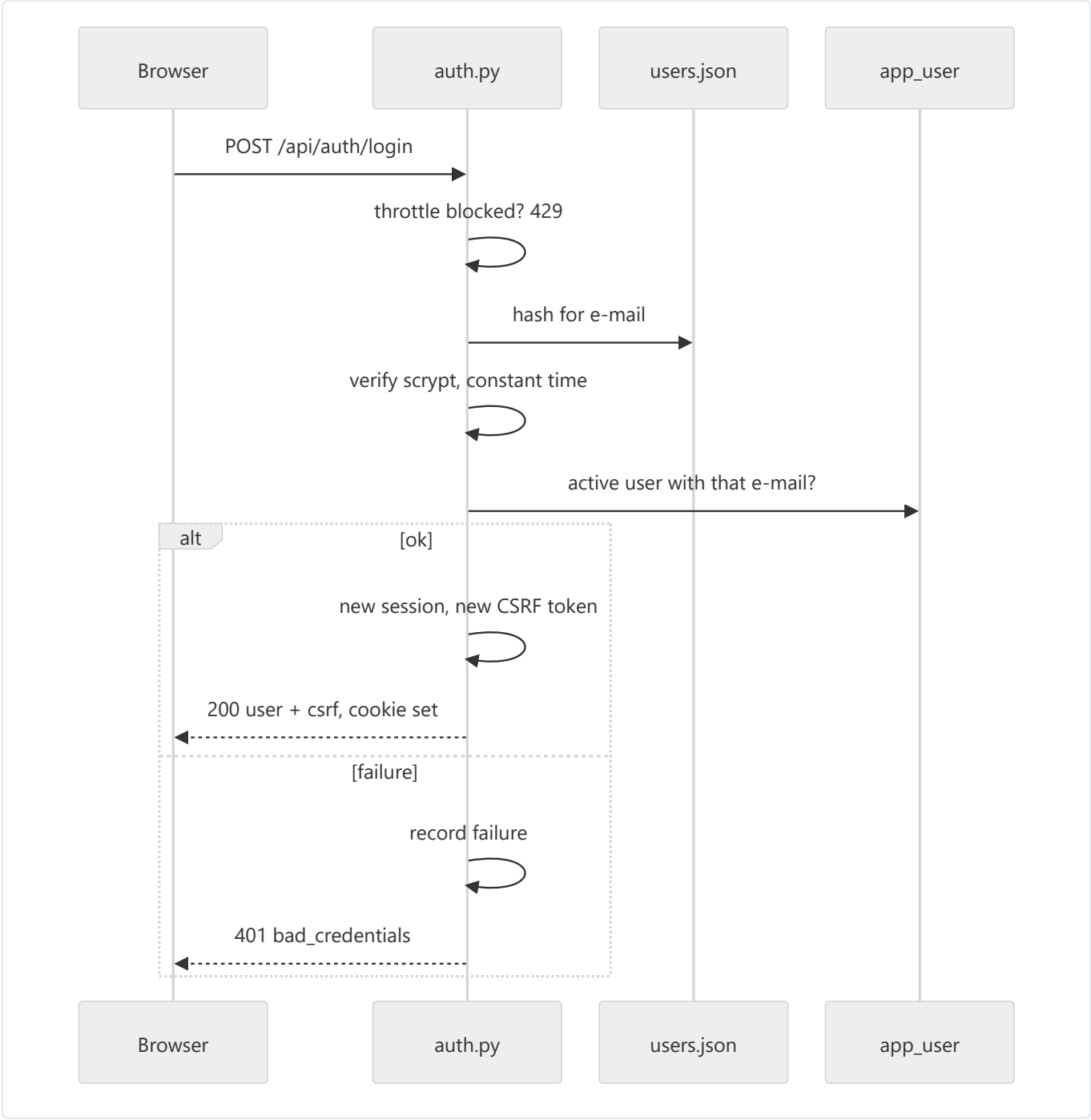
## 6.5 Document upload and download pipeline



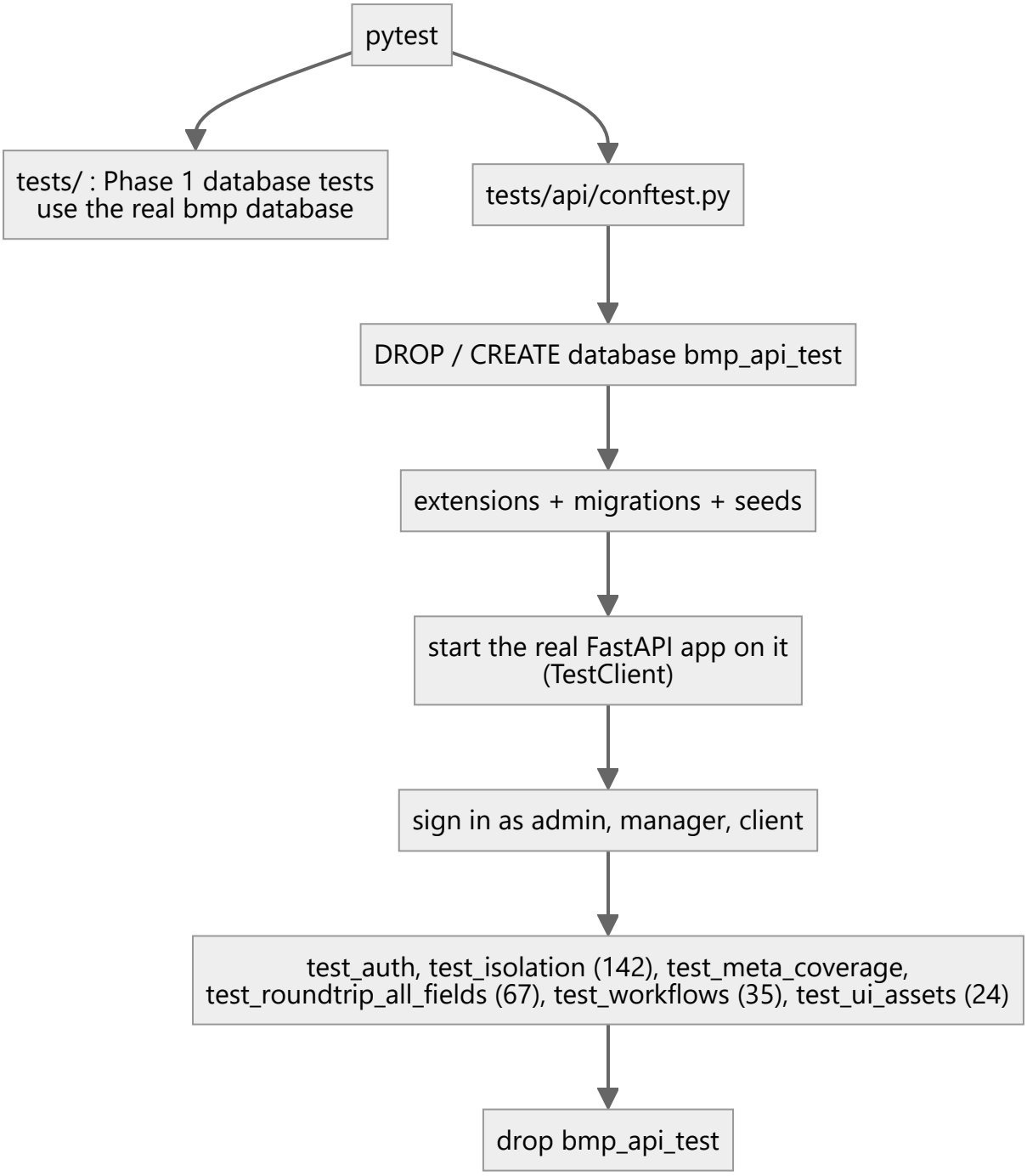

## 6.6 Audit pipeline

Every INSERT/UPDATE on a business table fires `audit.capture`, which stores the table, record id, operation, **actor** (`app.current_user_id`), time, old row and new row in `audit.log`. The log is append-only: updates, deletes and truncates are blocked, and the application login cannot even read it. The API reads it through the privileged connection, read-only, narrowed to companies the user may see: `GET /api/audit` (Activity Log screen) and `GET /api/entities/{key}/{pk}/history` (a record's History tab, showing field-by-field *from → to*).

## 6.7 Authentication pipeline



### 6.8 Test pipeline



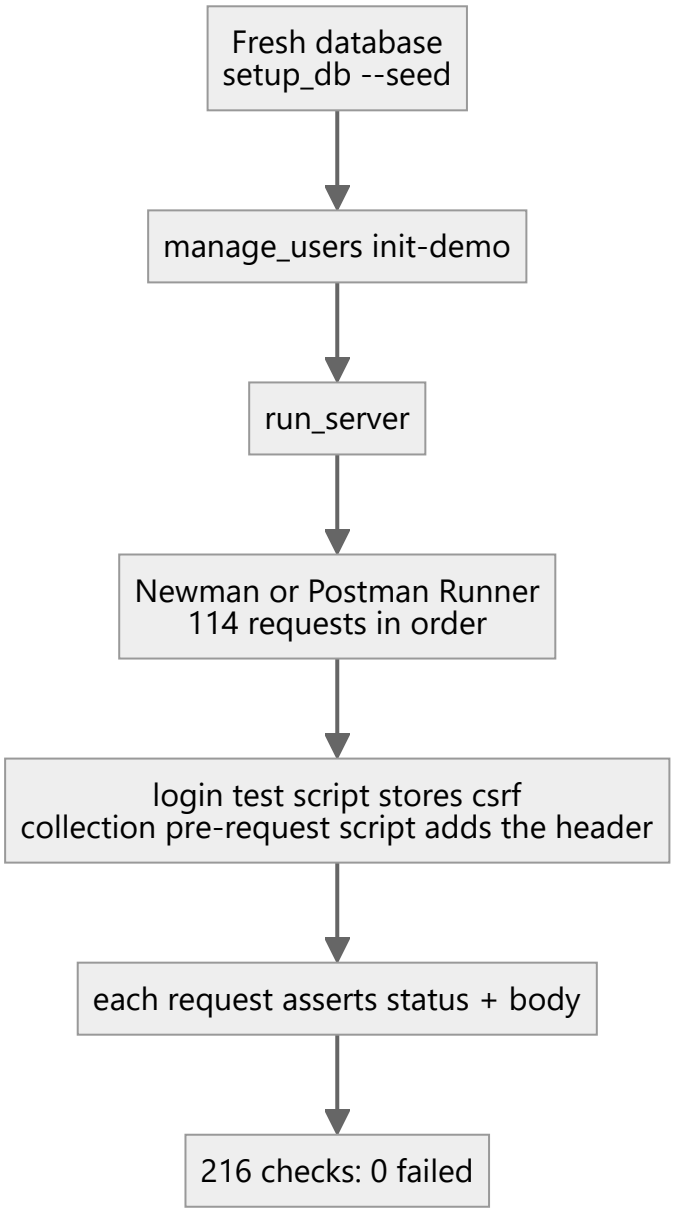

| Test file | Tests | What it proves |
|---|---|---|
| `tests/` (original) | 19 | The database: no deletes, overlaps rejected, audit, RLS, lookups, activity rules, traceability |
| `test_auth.py` | 11 | Sign-in, bad/unknown/inactive users, CSRF, lock-out, sign-out, password change |
| `test_meta_coverage.py` | 9 | Every table, column, key, dropdown and view is exposed; every table has a screen |
| `test_isolation.py` | 142 | For every table: a client sees only their company; a manager only theirs; people tables, child tables, views, audit, documents, exports, cross-company references |
| `test_roundtrip_all_fields.py` | 67 | Every writable field of every writable table stores exactly (a `Factory` in `payloads.py` fills them all); stale edits refused; archive/restore; validation messages; audit trail |
| `test_workflows.py` | 35 | Activities (due dates, order, confirmation, roll-up), meetings, register, documents (hash, version, tamper, bad files), exports, paging, secrets policy, rules, overview, administration |

| Test file | Tests | What it proves |
|---|---|---|
| `test_ui_assets.py` | 24 | Page and headers; no internet resources; approved CSS byte-identical; every JavaScript module parses ( `node --check` ) and every import and element id resolves; no unescaped database text |

## 6.9 Demo and traceability pipelines

- `run_demo.py` connects as `bmp_owner` and `bmp_app` and prints 10 numbered scenes proving the database's guarantees (history never overwritten, calculated values, no deletes, audit, activity engine ...). The "Rules as data" and "Isolation" scenes were removed on request.
- **Traceability**: `generate_traceability.py` maps every data row of *BMP_Master Checklist_Dashboard Requirements.xlsx* to a database object in `docs/FIELD_TRACEABILITY.md` ; `check_traceability.py` (and `tests/test_traceability.py` ) verify 100 % coverage.

## 6.10 API verification pipeline (Postman / Newman)



## 6.11 Recommended continuous-integration pipeline (proposal — not implemented, not tested)

The project has no CI configuration today. A reasonable first pipeline, using a PostgreSQL image that includes pgvector, would be:

```
# .github/workflows/ci.yml  -- PROPOSAL, untested; adapt names and secrets to your environment
name: ci
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg17
        env: { POSTGRES_PASSWORD: ci-admin-password }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres" --health-interval 5s --health-retries 10
    env:
      DATABASE_URL: postgresql+psycopg://bmp_owner:ci-owner@localhost:5432/bmp
      DATABASE_URL_APP: postgresql+psycopg://bmp_app:ci-app@localhost:5432/bmp
      DATABASE_URL_ADMIN: postgresql+psycopg://postgres:ci-admin-password@localhost:5432/postgres
      APP_ENV: development
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python scripts/setup_db.py --seed
      - run: python -m pytest -q
      - run: python scripts/check_traceability.py
```

A second job could start the server, seed the three demo passwords with `manage_users.py init-demo` and run Newman (2.8) as an end-to-end gate.

# PART 7 — Important core-logic files

This section explains the files where the product's behaviour lives, with the key code. Read these first when changing anything.

## 7. Core logic, file by file

### 7.1 `app/schema_meta.py` — the live schema reader (225 lines)

**Purpose:** build a `Catalog` of every table and view in schemas `core`, `activity`, `ops` by querying PostgreSQL itself, so the API never hard-codes a column list.

- `Column` — name, PostgreSQL type, nullability, default, generated/identity, comment, and a UI **`kind`** (`int`, `decimal`, `bool`, `date`, `time`, `datetime`, `json`, `bytea`, `vector`, `text`, `textarea`); plus `is_pk`, `fk`, `lookup` (Standard List code), `choices` (allowed values), `fmt` (financial year).
- Derived properties: `writable` (not generated, identity or a system column) and `required` (writable, `NOT NULL`, no default).
- `load_catalog(conn)` runs three queries: columns (`pg_attribute`), constraints (`pg_constraint`: primary, unique, foreign keys and CHECKs parsed by regex into allowed-value lists and the financial-year pattern), and `validate_lookup_*` triggers (which bind a column to a Standard List).

```
@property
def required(self) -> bool:
    """Must be supplied on create: NOT NULL with nothing to fall back on."""
    return self.writable and not self.nullable and self.default is None
```

**Change it when:** a new kind of database constraint should drive the forms. **Do not** add per-table code here.

### 7.2 `app/registry.py` — presentation and permissions per table (433 lines)

**Purpose:** the only hand-written knowledge about tables: the catalog gives the *shape*, the registry gives the *meaning*.

| Structure | Holds |
|---|---|
| `NAV` | The sidebar groups and items (`{"view": ...}` hand-built screens, `{"entity": ...}` generic pages) |
| `ENTITIES` | Per table: `label`, `singular`, `write` mode, `list` columns, `search` columns, `label_sql` (how one row prints as a sentence), `order`, `children` (related tables), `sections` (form groupings) |
| `SCOPE` | How a table is limited to a user's companies: `direct`, `parent`, `masters`, `global`, `admin`, `user` (see 7.4) |
| `VIEWS` | The 13 calculated views and their screen names |

Write modes: `company` (write/admin access to the row's company), `staff` (any user who can write somewhere — shared people records), `admin`, `user` (the person's own records, i.e. confirmations), `none`.

```
"company_rta": dict(label="Registrar & Transfer Agent", singular="RTA record", write="company",
    list=["name", "sebi_reg_no", "phone", "email", "valid_from", "valid_to"],
    search=["name", "sebi_reg_no"], label_sql="p.name", order="-valid_from"),
```

### 7.3 `app/crud.py` — the generic record engine (473 lines)

**Purpose:** list, get, options, create, update, archive/restore and view rows for **any** table, safely. `Engine` receives the `Catalog` and `Settings`.

| Method | What it does |
|---|---|
| `list(...)` | Builds one SQL statement from: row scope (read) + not-archived + company + financial year + `f_` column filters (typed through `coerce`) + escaped `ILIKE` search + validated sort. Paginates, then adds `_labels` (readable names for every foreign key, and for polymorphic `entity_type` / `entity_id`) and `_pk` |
| `get(...)` | One row inside the read scope, else 404 |
| `options(...)` | `(id, label)` pairs for dropdowns, using `label_sql` |
| `_prepare(...)` | Drops system/identity columns, rejects unknown fields, enforces the encrypted-column policy, coerces each value, and on create reports every missing required field at once |
| `_check_refs(...)` | Every foreign key must point to a row the user can see **and, for company-owned tables, in the same company**; lookup values must belong to the right Standard List |
| `create(...)` | Checks write permission and write access to the target company, validates, inserts, then proves the new row falls inside the user's write scope |
| `update(...)` | Same checks; primary keys and `company_id` cannot change; the `WHERE` clause includes the write scope **and** `updated_at = version`; if nothing changed, `_diagnose` decides between 404, 403 and 409 |
| `set_archived(...)` | Archive/restore; there is no delete method |
| `view_rows(...)` | Reads a calculated view and re-applies company scope (views bypass RLS) |

```
cur = conn.execute(f'UPDATE {t.ident} AS t SET {sets} WHERE {" AND ".join(where)}', params)
if cur.rowcount == 0:
    self._diagnose(conn, key, t, ctx, pkvals, version)   # 404 hidden / 403 read-only / 409 stale
return self.get(conn, key, ctx, pk)
```

## 7.4 `app/access.py` — who can see and change what (166 lines)

`UserCtx` carries the person's id, role and `access` map (`company_id → read | write | admin`), with `readable`, `writable`, `is_admin` and `is_staff`. `load_user` builds it from `core.app_user` and `core.user_company_access`. `Scope.row_scope(key, alias, ctx, mode, params)` returns a SQL fragment (with named parameters) for the rows the user may `read` or `write`:

| Scope type | Rule | Example tables |
|---|---|---|
| `direct` (default: has `company_id`) | `company_id` is one of the user's companies | meeting, government_filing, company itself (`id`) |
| `parent` | Via a parent row that is itself in scope | meeting_attendance → meeting; task_instance → activity_instance |
| `masters` | Shared people/holder records: staff see all; clients see only those linked to their companies; clients cannot write | director, shareholder |
| `global` | Everyone reads; only administrators write | lookup values, templates, rules, roles |
| `admin` | Administrators only; invisible (404) to others | app_user, user_company_access |
| `user` | A person's own rows (administrators see all) | confirmation |

`Scope.company_filter` narrows the same rules to one chosen company. Database RLS and these rules **must agree**; `test_isolation.py` loops over every table to prove it.

## 7.5 `app/coerce.py` — values in, types out (106 lines)

Turns JSON strings and numbers into typed Python/PostgreSQL values per column `kind`: blanks become `NULL`; booleans accept `true/yes/on/1`…; dates `YYYY-MM-DD`; date-times with an IST default; decimals checked against the column's precision (rounded to its scale); choices against the CHECK list; financial year against `YYYY-YY`; JSON and vector values parsed. It reports **form** problems only (`FieldError`); whether a combination is *allowed* stays with the database.

### 7.6 `app/errors.py` — one error type, friendly messages (107 lines)

`ApiError(status, code, message, fields)` plus helpers `bad_request` (422), `not_found`, `forbidden`, `conflict`. `map_db_error` translates PostgreSQL errors: unique → 409 `duplicate`; exclusion → 409 `overlap`; foreign key → 422; check → 422 with a plain-language sentence from a table of named constraints (for example *"DIN must be exactly 8 digits."*); not-null → 422 naming the field; `RAISE EXCEPTION` from the schema's own triggers → 422 with the trigger's own sentence; insufficient privilege → 403; connection problems → 503.

### 7.7 `app/security.py` and `app/deps.py` — identity (114 + 63 lines)

- `hash_password` / `verify_password` — scrypt with a per-password random salt; a missing user still burns the same work (no timing leak).
- `UsersStore` — reads/writes `instance/users.json` atomically (write to a temp file, replace), file mode 0600; `set_password` enforces ≥ 8 characters.
- `LoginThrottle` — in-memory failure counter per `address|email` (thread-safe).
- `csrf_ok` — constant-time compare of the session token and the `X-CSRF-Token` header.
- `deps.current_user` — the gate every protected route passes: needs `session["uid"]`; on non-GET requests checks CSRF; loads the user in a read-only transaction; clears the session if the account is no longer active. `admin_user`, `writable_company`, `readable_company`, `session`, `privileged_session` are the small building blocks the routers compose.

```
def current_user(request: Request) -> UserCtx:
    uid = request.session.get("uid")
    if not uid:
        raise ApiError(401, "unauthenticated", "Please sign in")
    if request.method not in SAFE_METHODS and not csrf_ok(
            request.session.get("csrf"), request.headers.get("x-csrf-token")):
        raise ApiError(403, "csrf", "Your session token is missing or out of date. Reload the page.")
    ...
```

### 7.8 `app/db.py` — the per-request transaction (51 lines)

```
@contextmanager
def tx(self, user_id: int | None = None, *, readonly: bool = False):
    """A transaction as `user_id`. Commits on success, rolls back on any exception."""
    with self.pool.connection() as conn:
        with conn.transaction():
            if readonly:
                conn.execute("SET TRANSACTION READ ONLY")
            conn.execute("SELECT set_config('app.current_user_id', %s, true)",
                         (str(user_id) if user_id else "",))
            yield conn
```

This 10-line method is what connects **application identity to database security**: the audit trigger, row-level security and the "no write without an actor" rule all read this one setting.

### 7.9 `app/routers/activities.py` — the workflow engine (315 lines)

Wraps the database's activity mechanics with the business rules: `_template_detail` (steps with dependencies, data bindings and evidence requirements), `_validate_params` (checks the start form against the template's `input_schema`), `start_activity` (calls `ops.instantiate_activity`), `_instance_detail` (tasks with blockers, links, evidence, confirmations, overdue flags), `task_status` (order, confirmation, evidence warning, roll-up — section 6.4), `instance_status`, `request_confirmation` and `decide_confirmation`.

## 7.10 Other logic worth knowing

| File | Key logic |
|---|---|
| `routers/people.py` | `add_to_board`: one transaction that **reuses** an existing person by DIN ("don't ask twice") or creates one, creates the board stint, and optionally the first designation. A second overlapping stint is refused by the database (409 `overlap`) |
| `routers/meetings.py` | `meeting_detail` computes the **roster** (directors on the board on the meeting date); `save_attendance` creates or updates one row per director, never deletes (set `attended=false` instead) |
| `routers/companies.py` | `overview` computes the tiles, "needs attention" list (overdue filings, rules needing action, expiring DSCs, overdue tasks, planned meetings), rule summary and per-group record counts; `create_company` runs on the privileged connection and grants the creator `admin` access in the same transaction; exports are scope-checked and formula-injection safe |
| `routers/views.py` | `calc_view` (scoped views); `rules` (applicability rules with the latest evaluation and two rules **computed** from the board's records: resident director, woman director) |
| `routers/documents.py` | Upload and download pipeline (6.5): streaming hash, versioning, link to evidence, integrity check |
| `routers/audit.py` | Company-narrowed read of `audit.log` on the privileged connection; skips `updated_at` / `created_at` noise |
| `routers/entities.py` | `BLOCK_CREATE` (company/activity/task must use their dedicated actions) and `LOCKED_FIELDS` (workflow-controlled fields); `_check_entity_reference` (evidence/task links must name a real, visible record) |
| `routers/meta.py` | Builds `/api/meta` once from catalog + registry; humanises names (`cin` → *CIN*); marks encrypted columns and whether policy allows them |

## 7.11 The database side (`db/`) — logic the API relies on

| Object | Behaviour |
|---|---|
| `audit.capture` (trigger) | Records every change with actor, time, old and new row; refuses a write when `app.current_user_id` is not set |
| `audit.block_delete`, `block_truncate`, `block_audit_log_mutation` | Nothing is ever deleted (63 tables; the two pure link tables `step_dependency` and `role_permission` are exempt); the audit log is immutable |
| `audit.touch_updated_at` | Maintains `updated_at` (the optimistic-lock version) |
| `audit.validate_lookup` | A lookup column must hold a value of its own Standard List |
| Exclusion constraints (`btree_gist`) | No overlapping periods for the same person/company/role |
| RLS policies (migration 011) | Company isolation on 32 tables via `app.current_user_id` |
| `ops.instantiate_activity` | Creates an activity instance and its tasks with computed due dates |
| `activity.get_template_json` | A template (steps, dependencies, bindings, evidence) as one JSON document — the shape a future AI/MCP tool would read |
| `activity.check_step_dependency`, `check_write_on_calculated` | Guard rails: a step may only depend on steps of the same template and never in a cycle; a step may not be bound in *write* mode to a calculated field |
| 13 `core.v_*` views | Ages, shareholding %, attendance %, overdue filings, rule checks, … never stored, so never stale |

# PART 8 — Testing the API with Postman

## 8. The Postman collection

### 8.1 Files

| File | Purpose |
|---|---|
| `docs/postman/BMP_Compliance_Platform.postman_collection.json` | The collection: **114 requests** in 14 folders, with **216 automatic checks** |
| `docs/postman/BMP_Compliance_Platform.postman_environment.json` | The environment: `baseUrl`, the three demo e-mails, three **password** variables (blank, marked secret), company ids |
| `docs/postman/sample_notice.pdf` | A tiny file for the upload test |

The same JSON is reproduced in full in Appendices A and B; the files are the authoritative copies (text copied out of a PDF can break at line ends).

### 8.2 Import and set up

1. Start the application (`python scripts/run_server.py`) with seeded data and the demo passwords set (`init-demo`).
2. Postman → **Import** → select the **collection** file and the **environment** file.
3. Top-right, choose the environment **BMP Compliance Platform - local**.
4. Open the environment and fill the three password variables (all three equal if you used `init-demo`). Leave the rest unless your database differs (see 8.3).
5. For the upload request, in folder **09 Documents and evidence** → **Upload a document…**, in the *Body* → *form-data* row `file`, click the field and pick `docs/postman/sample_notice.pdf` (or any small PDF/Word/Excel/image).
6. Open the collection → **Run** → run the whole collection in order (do not change the order: later folders reuse ids saved by earlier ones). Every request should show green checks.

### 8.3 Variables

| Variable | Where | Default | Meaning |
|---|---|---|---|
| `baseUrl` | env | `http://127.0.0.1:8000` | Server address |
| `adminEmail`, `managerEmail`, `clientEmail` | env | `admin@bmpco.in`, `manager@bmpco.in`, `client@nexatech.co.in` | Demo users |
| `adminPassword`, `managerPassword`, `clientPassword` | env (secret) | *blank* | **You fill these in** |
| `companyId` | env | `2` | The main working company (BrightMed Pharma) |
| `otherCompanyId` | env | `3` | A second company the administrator can see |
| `clientCompanyId` | env | `1` | The client's own company (Nexatech) |
| `csrf` | collection (script) | blank | Set by every login request; sent as `X-CSRF-Token` by the collection pre-request script |

| Variable | Where | Default | Meaning |
|---|---|---|---|
| `meetingTypeId`, `designationId`, `boardCategoryId`, `directorId`, `ruleId`, `templateId`, `templateCode`, `instanceId`, `taskId1…9`, `confirmationId`, `rtaId`, `meetingId`, `documentId`, … | collection (script) | blank | Ids captured from earlier responses |

> If your database was not built from the standard seed, check `GET /api/companies` and set `companyId`, `otherCompanyId` and `clientCompanyId` to real ids.

## 8.4 How sign-in, cookies and CSRF are handled

- **Cookies:** Postman keeps the `bmp_session` cookie automatically after *Login*.
- **CSRF:** each Login request's test script saves `csrf` from the response. A **collection-level pre-request script** then adds `X-CSRF-Token: {{csrf}}` to every request that is not GET/HEAD and is not the login itself. A few negative tests override the header on purpose (for example *a change with a wrong CSRF token → 403*).
- **Roles:** folder 12 signs in as the read-only client, runs the isolation checks, signs out and signs back in as administrator; folder 13 then signs out and proves the session is gone.

## 8.5 What the collection proves

- **Every route is exercised** (see the coverage table below), both the success path and, where meaningful, the refusal path: 401 not signed in, 403 CSRF / read-only, 404 hidden record, 409 stale / blocked / overlap, 422 validation with named fields, 405 no DELETE.
- The **full activity lifecycle** (start → order rule → completion → confirmation → skip → cancel → reopen).
- **Company isolation** from the client's point of view.
- The collection **creates test records** (an RTA, a meeting, a director, an activity, a document, a user, a company). Nothing is deleted; the test company and test user are archived at the end of their folders, the other test records stay visible in the lists. It is safe to re-run: it was run twice in a row on the same database with no failures.

## 8.6 Route coverage

**All 43 routes are exercised** (every router endpoint, the health probe and the page).

| Method | Path | Exercised in folder(s) |
|---|---|---|
| **POST** | `/api/auth/login` | 01, 12 |
| **POST** | `/api/auth/logout` | 12, 13 |
| **GET** | `/api/auth/me` | 01, 13 |
| **POST** | `/api/auth/password` | 01 |
| **POST** | `/api/admin/users/{user_id}/password` | 11 |
| **GET** | `/api/meta` | 02 |
| **GET** | `/api/options/{entity}` | 02 |
| **GET** | `/api/entities/{key}` | 04, 11, 12 |
| **GET** | `/api/entities/{key}/export.csv` | 04 |
| **GET** | `/api/entities/{key}/{pk}` | 04, 08, 09 |
| **POST** | `/api/entities/{key}` | 01, 03, 04, 08, 11, 12 |
| **PATCH** | `/api/entities/{key}/{pk}` | 03, 04, 08 |
| **POST** | `/api/entities/{key}/{pk}/archive` | 03, 04, 11 |
| **POST** | `/api/entities/{key}/{pk}/restore` | 04 |
| **GET** | `/api/entities/{key}/{pk}/history` | 04 |

| Method | Path | Exercised in folder(s) |
|---|---|---|
| **GET** | `/api/companies` | 03, 12 |
| **POST** | `/api/companies` | 03 |
| **GET** | `/api/companies/{company_id}/years` | 03 |
| **GET** | `/api/companies/{company_id}/overview` | 03, 12 |
| **GET** | `/api/companies/{company_id}/export.json` | 03 |
| **GET** | `/api/companies/{company_id}/export.xlsx` | 03, 12 |
| **GET** | `/api/companies/{company_id}/directors-register` | 05 |
| **POST** | `/api/companies/{company_id}/directors-register` | 05 |
| **GET** | `/api/companies/{company_id}/meetings-book` | 06 |
| **GET** | `/api/meetings/{meeting_id}/detail` | 06 |
| **PUT** | `/api/meetings/{meeting_id}/attendance` | 06 |
| **GET** | `/api/calc/{view}` | 07, 12 |
| **GET** | `/api/companies/{company_id}/rules` | 03 |
| **GET** | `/api/activities/templates` | 08 |
| **GET** | `/api/activities/templates/{template_id}` | 08 |
| **GET** | `/api/activities/templates/{code}/json` | 08 |
| **POST** | `/api/activities/instances` | 08, 12 |
| **GET** | `/api/companies/{company_id}/activities` | 08 |
| **GET** | `/api/activities/instances/{instance_id}` | 08 |
| **POST** | `/api/activities/tasks/{task_id}/status` | 08 |
| **POST** | `/api/activities/instances/{instance_id}/status` | 08 |
| **POST** | `/api/activities/tasks/{task_id}/confirmations` | 08 |
| **POST** | `/api/confirmations/{confirmation_id}/{decision}` | 08 |
| **POST** | `/api/documents/upload` | 09 |
| **GET** | `/api/documents/{document_id}/download` | 09 |
| **GET** | `/api/audit` | 10 |
| **GET** | `/api/health` | 00 |
| **GET** | `/` | 00 |

## 8.7 Request catalogue

Expected status is shown after the arrow; *Checks* is the number of assertions in that request.

### 00 Health and page

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Health check (public) | **GET** | `/api/health` | 200 | 2 |
| 2 | Dashboard page (public) | **GET** | `/` | 200 | 3 |

### 01 Authentication and session

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Not signed in -> 401 | **GET** | `/api/auth/me` | 401 | 2 |
| 2 | Login with a wrong password -> 401 | **POST** | `/api/auth/login` | 401 | 2 |

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 3 | Login as administrator | **POST** | `/api/auth/login` | 200 | 3 |
| 4 | Who am I | **GET** | `/api/auth/me` | 200 | 3 |
| 5 | Change my password (to the same value) | **POST** | `/api/auth/password` | 200 | 1 |
| 6 | Change my password - wrong current -> 403 | **POST** | `/api/auth/password` | 403 | 1 |
| 7 | A change without the CSRF token -> 403 (see note) | **POST** | `/api/entities/company_rta` | 403 | 2 |

### 02 Metadata (the self-describing API)

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Get all metadata | **GET** | `/api/meta` | 200 | 6 |
| 2 | Dropdown options - board stints of a company | **GET** | `/api/options/directorship` ?company_id={{companyId}}&limit=20 | 200 | 2 |
| 3 | Dropdown options - search by text | **GET** | `/api/options/director` ?q=joshi | 200 | 2 |
| 4 | Dropdown options - fetch by ids | **GET** | `/api/options/director` ?ids={{directorId}} | 200 | 2 |

### 03 Companies

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | List my companies | **GET** | `/api/companies` | 200 | 2 |
| 2 | Financial years that have data | **GET** | `/api/companies/{{companyId}}/years` | 200 | 2 |
| 3 | Overview (tiles, attention, rules, segments) | **GET** | `/api/companies/{{companyId}}/overview` ?fy=2024-25 | 200 | 2 |
| 4 | Applicability rules for a year | **GET** | `/api/companies/{{companyId}}/rules` ?fy=2024-25 | 200 | 2 |
| 5 | Record a rule evaluation (new dated record) | **POST** | `/api/entities/rule_evaluation` | 201 | 2 |
| 6 | Export one company (Excel) | **GET** | `/api/companies/{{companyId}}/export.xlsx` | 200 | 2 |
| 7 | Export one company (JSON) | **GET** | `/api/companies/{{companyId}}/export.json` | 200 | 2 |
| 8 | Add a company (administrator) | **POST** | `/api/companies` | 201 | 2 |
| 9 | Edit that company (version-checked) | **PATCH** | `/api/entities/company/{{newCompanyId}}` | 200 | 2 |
| 10 | Archive that company (nothing is deleted) | **POST** | `/api/entities/company/{{newCompanyId}}/archive` | 200 | 2 |

### 04 Generic records (any table) - full lifecycle

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Create a record (Registrar & Transfer Agent) | **POST** | `/api/entities/company_rta` | 201 | 3 |
| 2 | List with search, sort and paging | **GET** | `/api/entities/company_rta` ?company_id={{companyId}}&q=Postman&sort=valid_from&desc=true&page=1&size=10 | 200 | 2 |
| 3 | List filtered by a column and financial year | **GET** | `/api/entities/meeting` ?company_id={{companyId}}&fy=2024-25&f_status=held | 200 | 2 |
| 4 | Get one record | **GET** | `/api/entities/company_rta/{{rtaId}}` | 200 | 2 |
| 5 | Edit with the version you last saw | **PATCH** | `/api/entities/company_rta/{{rtaId}}` | 200 | 2 |

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 6 | Edit with a stale version -> 409 | **PATCH** | `/api/entities/company_rta/{{rtaId}}` | 409 | 2 |
| 7 | Validation errors name the fields -> 422 | **POST** | `/api/entities/company_rta` | 422 | 2 |
| 8 | A database rule in plain language -> 422 | **POST** | `/api/entities/company_business_activity` | 422 | 2 |
| 9 | Record history (who, when, from -> to) | **GET** | `/api/entities/company_rta/{{rtaId}}/history` | 200 | 2 |
| 10 | Archive (hide, never delete) | **POST** | `/api/entities/company_rta/{{rtaId}}/archive` | 200 | 2 |
| 11 | Archived record appears only when asked | **GET** | `/api/entities/company_rta` ?company_id={{companyId}}&q=Postman&archived=true | 200 | 2 |
| 12 | Restore | **POST** | `/api/entities/company_rta/{{rtaId}}/restore` | 200 | 2 |
| 13 | Export a list as CSV | **GET** | `/api/entities/company_rta/export.csv` ?company_id={{companyId}} | 200 | 2 |
| 14 | DELETE is not offered -> 405 | **DELETE** | `/api/entities/company_rta/{{rtaId}}` | 405 | 1 |
| 15 | Unknown record type -> 404 | **GET** | `/api/entities/not_a_table` | 404 | 2 |
| 16 | Create a meeting (lookup + dates + enum) | **POST** | `/api/entities/meeting` | 201 | 2 |
| 17 | A lookup from the wrong list -> 422 | **POST** | `/api/entities/meeting` | 422 | 2 |
| 18 | Mark the meeting as held | **PATCH** | `/api/entities/meeting/{{meetingId}}` | 200 | 2 |
| 19 | Add an agenda item to it (child record) | **POST** | `/api/entities/meeting_agenda_item` | 201 | 1 |

### 05 Directors and KMP register

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Board register (every stint, current and ceased) | **GET** | `/api/companies/{{companyId}}/directors-register` | 200 | 2 |
| 2 | Register - current directors only | **GET** | `/api/companies/{{companyId}}/directors-register` ?include_ceased=false | 200 | 2 |
| 3 | Add a NEW person to the board in one step | **POST** | `/api/companies/{{companyId}}/directors-register` | 201 | 2 |
| 4 | The same DIN on another company REUSES the person | **POST** | `/api/companies/{{otherCompanyId}}/directors-register` | 201 | 2 |
| 5 | Overlapping stint is refused -> 409 | **POST** | `/api/companies/{{companyId}}/directors-register` | 409 | 2 |
| 6 | Missing appointment date -> 422 | **POST** | `/api/companies/{{companyId}}/directors-register` | 422 | 1 |

### 06 Meetings

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Meetings book for a year | **GET** | `/api/companies/{{companyId}}/meetings-book` ?fy=2024-25 | 200 | 2 |
| 2 | One meeting in full (with the board roster) | **GET** | `/api/meetings/{{seedMeetingId}}/detail` | 200 | 2 |
| 3 | Save attendance by roster (create or update, never duplicate) | **PUT** | `/api/meetings/{{seedMeetingId}}/attendance` | 200 | 2 |

### 07 Calculated values (database views)

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | v_current_directors | **GET** | `/api/calc/v_current_directors` ?company_id={{companyId}} | 200 | 2 |
| 2 | v_director_history | **GET** | `/api/calc/v_director_history` ?company_id={{companyId}} | 200 | 2 |
| 3 | v_director_age | **GET** | `/api/calc/v_director_age` ?company_id={{companyId}} | 200 | 2 |
| 4 | v_shareholding_percent | **GET** | `/api/calc/v_shareholding_percent` ?company_id={{companyId}} | 200 | 2 |
| 5 | v_top10_shareholders | **GET** | `/api/calc/v_top10_shareholders` ?company_id={{companyId}} | 200 | 2 |
| 6 | v_company_shareholder_counts | **GET** | `/api/calc/v_company_shareholder_counts` ?company_id={{companyId}} | 200 | 2 |
| 7 | v_director_attendance_summary | **GET** | `/api/calc/v_director_attendance_summary` ?company_id={{companyId}} | 200 | 2 |
| 8 | v_committee_attendance_percent | **GET** | `/api/calc/v_committee_attendance_percent` ?company_id={{companyId}} | 200 | 2 |
| 9 | v_independent_director_years | **GET** | `/api/calc/v_independent_director_years` ?company_id={{companyId}} | 200 | 2 |
| 10 | v_s186_running_total | **GET** | `/api/calc/v_s186_running_total` ?company_id={{companyId}} | 200 | 2 |
| 11 | v_filings_overdue | **GET** | `/api/calc/v_filings_overdue` ?company_id={{companyId}} | 200 | 2 |
| 12 | v_rule_resident_director | **GET** | `/api/calc/v_rule_resident_director` ?company_id={{companyId}} | 200 | 2 |
| 13 | v_rule_woman_director | **GET** | `/api/calc/v_rule_woman_director` ?company_id={{companyId}} | 200 | 2 |
| 14 | Unknown view -> 404 | **GET** | `/api/calc/not_a_view` | 404 | 1 |

### 08 Activities and tasks

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | List activity templates | **GET** | `/api/activities/templates` | 200 | 2 |
| 2 | Template detail (steps, bindings, evidence) | **GET** | `/api/activities/templates/{{templateId}}` | 200 | 2 |
| 3 | Template as one JSON document (the shape a future MCP tool will use) | **GET** | `/api/activities/templates/{{templateCode}}/json` | 200 | 2 |
| 4 | Start without required fields -> 422 | **POST** | `/api/activities/instances` | 422 | 2 |
| 5 | Start a Board Meeting activity (creates 9 tasks with due dates) | **POST** | `/api/activities/instances` | 201 | 2 |
| 6 | This company's activities | **GET** | `/api/companies/{{companyId}}/activities` | 200 | 2 |
| 7 | One activity with tasks, evidence and confirmations | **GET** | `/api/activities/instances/{{instanceId}}` | 200 | 2 |
| 8 | Complete step 2 first -> 409 blocked | **POST** | `/api/activities/tasks/{{taskId2}}/status` | 409 | 2 |
| 9 | Step 1 -> in progress | **POST** | `/api/activities/tasks/{{taskId1}}/status` | 200 | 2 |
| 10 | Step 1 -> completed | **POST** | `/api/activities/tasks/{{taskId1}}/status` | 200 | 2 |
| 11 | Step 2 -> completed | **POST** | `/api/activities/tasks/{{taskId2}}/status` | 200 | 1 |
| 12 | Step 3 -> completed (warns about missing evidence) | **POST** | `/api/activities/tasks/{{taskId3}}/status` | 200 | 2 |

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 13 | Step 4 needs a confirmation -> 409 | **POST** | `/api/activities/tasks/{{taskId4}}/status` | 409 | 2 |
| 14 | Request the confirmation | **POST** | `/api/activities/tasks/{{taskId4}}/confirmations` | 201 | 2 |
| 15 | Confirm it (an administrator may confirm their own request) | **POST** | `/api/confirmations/{{confirmationId}}/confirm` | 200 | 2 |
| 16 | Step 4 -> completed (now allowed) | **POST** | `/api/activities/tasks/{{taskId4}}/status` | 200 | 1 |
| 17 | Skip step 5 | **POST** | `/api/activities/tasks/{{taskId5}}/status` | 200 | 1 |
| 18 | Cancel the activity | **POST** | `/api/activities/instances/{{instanceId}}/status` | 200 | 2 |
| 19 | Nothing moves while cancelled -> 409 | **POST** | `/api/activities/tasks/{{taskId6}}/status` | 409 | 1 |
| 20 | Reopen the activity | **POST** | `/api/activities/instances/{{instanceId}}/status` | 200 | 2 |
| 21 | Edit a task note (the only directly editable task field) | **GET** | `/api/entities/task_instance/{{taskId1}}` | 200 | 1 |
| 22 | Save the note | **PATCH** | `/api/entities/task_instance/{{taskId1}}` | 200 | 2 |
| 23 | Task status cannot be PATCHed directly -> 422 | **PATCH** | `/api/entities/task_instance/{{taskId1}}` | 422 | 1 |
| 24 | Link a record to a task | **POST** | `/api/entities/task_record_link` | 201 | 2 |

### 09 Documents and evidence

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Upload a document and link it to a meeting | **POST** | `/api/documents/upload` | 201 | 2 |
| 2 | Download it again (integrity re-checked) | **GET** | `/api/documents/{{documentId}}/download` | 200 | 2 |
| 3 | The document's record | **GET** | `/api/entities/document/{{documentId}}` | 200 | 2 |
| 4 | Upload with no file -> 422 | **POST** | `/api/documents/upload` | 422 | 1 |

### 10 Audit trail

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Activity log for a company | **GET** | `/api/audit` ?company_id={{companyId}}&limit=20 | 200 | 2 |
| 2 | Everything I may see (filtered by table) | **GET** | `/api/audit` ?table=meeting&limit=10 | 200 | 2 |

### 11 Administration (administrator only)

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | List users | **GET** | `/api/entities/app_user` ?q=client | 200 | 2 |
| 2 | Set a user's sign-in password | **POST** | `/api/admin/users/{{clientUserId}}/password` | 200 | 1 |
| 3 | Create a user | **POST** | `/api/entities/app_user` | 201 | 1 |
| 4 | Grant that user read access to a company | **POST** | `/api/entities/user_company_access` | 201 | 1 |
| 5 | Archive the user (users are never deleted) | **POST** | `/api/entities/app_user/{{newUserId}}/archive` | 200 | 1 |
| 6 | List Standard List values | **GET** | `/api/entities/lookup_value` ?size=5 | 200 | 2 |

### 12 Isolation checks (signs in as the read-only client, then back as admin)

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Login as the read-only client (Nexatech only) | **POST** | `/api/auth/login` | 200 | 3 |
| 2 | Client sees only their company | **GET** | `/api/companies` | 200 | 3 |
| 3 | Another company's overview -> 404 | **GET** | `/api/companies/{{companyId}}/overview` | 404 | 1 |
| 4 | Another company's export -> 404 | **GET** | `/api/companies/{{companyId}}/export.xlsx` | 404 | 1 |
| 5 | Administration tables are hidden -> 404 | **GET** | `/api/entities/app_user` | 404 | 1 |
| 6 | A read-only user cannot write -> 403 | **POST** | `/api/entities/company_rta` | 403 | 2 |
| 7 | Calculated views are narrowed to their company | **GET** | `/api/calc/v_current_directors` | 200 | 2 |
| 8 | People are limited to those linked to their company | **GET** | `/api/entities/director` | 200 | 2 |
| 9 | Client cannot start an activity -> 403 | **POST** | `/api/activities/instances` | 403 | 1 |
| 10 | Sign out the client | **POST** | `/api/auth/logout` | 200 | 1 |
| 11 | Sign back in as administrator | **POST** | `/api/auth/login` | 200 | 3 |

**13 Sign out**

| # | Request | Method | Path | Expect | Checks |
|---|---|---|---|---|---|
| 1 | Logout | **POST** | `/api/auth/logout` | 200 | 2 |
| 2 | Session is gone -> 401 | **GET** | `/api/auth/me` | 401 | 1 |

## 8.8 Verification performed for this guide

The collection was validated with the real **Newman** runner (not just imported): a throw-away database `bmp_postman` was built from the project's own migrations and seeds, the server was started against it, and the collection was run with Newman.

| Run | Requests | Checks | Failed |
|---|---|---|---|
| First complete run on a fresh database | 114 | 216 | 0 |
| Second run on the *same* database (repeatability) | 114 | 216 | 0 |

While preparing the run, one environment problem was found and fixed outside the product: a deep scratch folder made the stored file's full path exceed Windows' 260-character limit, so the upload returned 500 until `STORAGE_DIR` was shortened (see the last row of table 2.10).

## 8.9 Limits and tips

- The collection relies on the **seed data** (people, rules, one seeded Board Meeting template, meeting `BM-1`). A database that lacks them will fail the checks that look them up.
- One test deliberately signs in with a wrong password; with `LOGIN_MAX_FAILURES=5` this never reaches the lock-out in a normal run.
- **Newman:** pass `--working-dir` pointing at `docs/postman` so `sample_notice.pdf` is found.
- To add a request: copy a sibling, keep the `Status is …` check, and save any id you need with a line in the test script (`pm.collectionVariables.set('myId', j.row.id)`).

# PART 9 — Quick answers

## 9. Frequently asked questions

| Question | Answer |
|---|---|
| Where do passwords live? | Hashes in `instance/users.json`; the database has no password column by design |
| Why can't I delete a record? | Nothing is ever deleted. Use **Archive** (undo with **Restore**); the database also blocks deletes |
| Who can see which company? | Rows in `core.user_company_access` (read / write / admin); enforced by RLS and by the API's `Scope` |
| How do I add a new person to the system? | Administration → Users, then User company access, then `manage_users.py set-password` (or *Set password* in the record) |
| How do I add a company? | Companies page → **Add company** (administrators). It runs on the privileged connection and gives the creator admin access |
| How do two people editing the same record avoid overwriting each other? | The edit carries `updated_at`; the second save gets 409 `stale` |
| Where are uploaded files? | `storage/<company_id>/<first two hex of SHA-256>/<sha256>_<name>`; back up with the database |
| How do I run only the API tests quickly? | `python -m pytest tests/api -q` |
| Is it ready for production? | See `BMP_UI_Backend_Implementation_Guide.md` Part D for the recommended hardening list (HTTPS reverse proxy, secrets, backups, CI) |

# Appendix A — Postman environment (JSON)

Save as `BMP_Compliance_Platform.postman_environment.json` (or use the file in `docs/postman`). The three password values are intentionally blank.


```
{
  "id": "f8157033-cc05-5909-87dc-5ee040a0086e",
  "name": "BMP Compliance Platform - local",
  "values": [
    {"key": "baseUrl", "value": "http://127.0.0.1:8000", "type": "default", "enabled": true},
    {"key": "adminEmail", "value": "admin@bmpco.in", "type": "default", "enabled": true},
    {"key": "adminPassword", "value": "", "type": "secret", "enabled": true},
    {"key": "managerEmail", "value": "manager@bmpco.in", "type": "default", "enabled": true},
    {"key": "managerPassword", "value": "", "type": "secret", "enabled": true},
    {"key": "clientEmail", "value": "client@nexatech.co.in", "type": "default", "enabled": true},
    {"key": "clientPassword", "value": "", "type": "secret", "enabled": true},
    {"key": "companyId", "value": "2", "type": "default", "enabled": true},
    {"key": "otherCompanyId", "value": "3", "type": "default", "enabled": true},
    {"key": "clientCompanyId", "value": "1", "type": "default", "enabled": true}
  ],
  "_postman_variable_scope": "environment"
}
```

# Appendix B — Postman collection (JSON)

Save as `BMP_Compliance_Platform.postman_collection.json` (or use the file in `docs/postman`; recommended — text copied from a PDF can be broken by line wrapping). It is shown in full below, folder by folder.


```
{
  "info": {
    "_postman_id": "c116a69e-fc8f-55dd-8597-7b9e2b6466a9",
    "name": "BMP Compliance Platform API",
    "description": "Tests every endpoint of the BMP Compliance Platform API.\n\n**Run order:** use the Collection Runner on the whole collection (folders
00-13 are in dependency order), or run `01 > Login as administrator` first and then any single request.\n\n**Environment:** import
`BMP_Compliance_Platform.postman_environment.json` and set the three passwords to the one you gave `python scripts/manage_users.py init-
demo`.\n\n**Data:** assumes the seeded demo data (company 2 = BrightMed, company 1 = Nexatech). It creates test records; nothing is ever deleted.\n\nEvery
request has test scripts, so a green run proves the API works.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "event": [
    {
      "listen": "prerequest",
      "script": {
        "type": "text/javascript",
        "exec": ["// Every change must carry the CSRF token returned by login.","const m = pm.request.method;","const token =
pm.collectionVariables.get('csrf');","if (m !== 'GET' && m !== 'HEAD' && token && !pm.request.headers.has('X-CSRF-Token') &&
!pm.request.url.getPath().endsWith('/auth/login')) {","  pm.request.headers.add({ key: 'X-CSRF-Token', value: token });","}"]
      }
    }
  ],
  "variable": [
    {
      "key": "baseUrl",
      "value": "http://127.0.0.1:8000"
    },
    {
      "key": "adminEmail",
      "value": "admin@bmpco.in"
    },
    {
      "key": "adminPassword",
      "value": ""
    },
    {
      "key": "managerEmail",
      "value": "manager@bmpco.in"
    },
    {
      "key": "managerPassword",
      "value": ""
    },
    {
      "key": "clientEmail",
      "value": "client@nexatech.co.in"
    },
    {
      "key": "clientPassword",
      "value": ""
    },
    {
      "key": "companyId",
      "value": "2"
    },
    {
      "key": "otherCompanyId",
      "value": "3"
    },
    {
      "key": "clientCompanyId",
      "value": "1"
    },
    {
      "key": "csrf",
      "value": ""
    }
  ],
  "item": [
    {
      "name": "00 Health and page",
      "item": [
        {"name":"Health check (public)","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/health","host":["{{baseUrl}}"],"path":
["api","health"]},"description":"No sign-in needed. Use it for monitoring."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","pm.test('Database is up', () =>
pm.expect(pm.response.json().database).to.eql(true));"]}}]},
        {"name":"Dashboard page (public)","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/","host":["{{baseUrl}}"],"path":[]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","pm.test('HTML is served',
() => pm.expect(pm.response.text()).to.include('Compliance Dashboard'));","pm.test('Security headers present', () => {
pm.expect(pm.response.headers.get('X-Frame-Options')).to.eql('DENY'); pm.expect(pm.response.headers.get('Content-Security-Policy')).to.include(\"default-
src 'self'\"); });"]}}]}
      ]
    },
    {
      "name": "01 Authentication and session",
      "item": [
        {"name":"Not signed in -> 401","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/auth/me","host":["{{baseUrl}}"],"path":
["api","auth","me"]},"description":"Run this before logging in (or after logging out) to see the protection."},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 401', () => pm.response.to.have.status(401));","pm.test('Error code is unauthenticated', () =>
pm.expect(pm.response.json().error.code).to.eql('unauthenticated'));"]}}]},
        {"name":"Login with a wrong password -> 401","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/login","host":["
{{baseUrl}}"],"path":["api","auth","login"]},"body":{"mode":"raw","raw":"{\n  \"email\": \"{{adminEmail}}\",\n  \"password\": \"definitely-
```

```
wrong\"\n}","options":{"raw":{"language":"json"}}},"description":"Counts towards the lock-out (5 failures in 10 minutes). Run it once."},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 401', () => pm.response.to.have.status(401));","pm.test('Error code is
bad_credentials', () => pm.expect(pm.response.json().error.code).to.eql('bad_credentials'));"]}}]},
        {"name":"Login as administrator","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/login","host":["{{baseUrl}}"],"path":
["api","auth","login"]},"body":{"mode":"raw","raw":"{\n  \"email\": \"{{adminEmail}}\",\n  \"password\": \"{{adminPassword}}\"\n}","options":{"raw":
{"language":"json"}}},"description":"Signs in. The test script stores the returned CSRF token in the collection variable `csrf`; the collection's pre-
request script then sends it as the `X-CSRF-Token` header on every change. The session cookie is kept by Postman's cookie jar."},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.csrf); if (v !== undefined && v !== null) pm.collectionVariables.set('csrf',
String(v)); } catch (e) {}","pm.test('Returns the user and a CSRF token', () => { pm.expect(j.csrf).to.be.a('string').and.not.empty;
pm.expect(j.user.email).to.be.a('string'); });","pm.test('is_admin is true', () => pm.expect(j.user.is_admin).to.eql(true));"]}}]},
        {"name":"Who am I","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/auth/me","host":["{{baseUrl}}"],"path":
["api","auth","me"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.user.id); if (v !==
undefined && v !== null) pm.collectionVariables.set('adminUserId', String(v)); } catch (e) {}","pm.test('Signed in as the administrator', () =>
pm.expect(j.user.email).to.eql(pm.variables.get('adminEmail')));","pm.test('Has access to companies', () =>
pm.expect(Object.keys(j.user.access).length).to.be.above(0));"]}}]},
        {"name":"Change my password (to the same value)","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/password","host":["
{{baseUrl}}"],"path":["api","auth","password"]},"body":{"mode":"raw","raw":"{\n  \"current_password\": \"{{adminPassword}}\",\n  \"new_password\": \"
{{adminPassword}}\"\n}","options":{"raw":{"language":"json"}}},"description":"Safe to run: the new password equals the current one. Needs at least 8
characters."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));"]}}]},
        {"name":"Change my password - wrong current -> 403","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/password","host":["
{{baseUrl}}"],"path":["api","auth","password"]},"body":{"mode":"raw","raw":"{\n  \"current_password\": \"wrong\",\n  \"new_password\":
\"AnotherPassw0rd!\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is
403', () => pm.response.to.have.status(403));"]}}]},
        {"name":"A change without the CSRF token -> 403 (see note)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta","host":["{{baseUrl}}"],"path":["api","entities","company_rta"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"
{{companyId}}\",\n  \"name\": \"x\",\n  \"valid_from\": \"2024-04-01\"\n}","options":{"raw":{"language":"json"}}},"description":"The pre-request script
deliberately overrides the token with a wrong one to demonstrate CSRF protection."},"event":[{"listen":"prerequest","script":
{"type":"text/javascript","exec":["pm.request.headers.upsert({ key: 'X-CSRF-Token', value: 'wrong-token' });"]}},{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 403', () => pm.response.to.have.status(403));","pm.test('Error code is csrf', () =>
pm.expect(pm.response.json().error.code).to.eql('csrf'));"]}}]}
      ]
    },
    {
      "name": "02 Metadata (the self-describing API)",
      "item": [
        {"name":"Get all metadata","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/meta","host":["{{baseUrl}}"],"path":
["api","meta"]},"description":"Everything the UI needs to draw itself: entities, fields, dropdown lists, navigation, calculated views, policy
flags."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j =
{}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.lookups.MEETING_TYPE.find(x => x.code === 'BOARD').id ||
j.lookups.MEETING_TYPE[0].id); if (v !== undefined && v !== null) pm.collectionVariables.set('meetingTypeId', String(v)); } catch (e) {}","try { const v =
(j.lookups.DESIGNATION[0].id); if (v !== undefined && v !== null) pm.collectionVariables.set('designationId', String(v)); } catch (e) {}","try { const v =
(j.lookups.DIRECTOR_CATEGORY[0].id); if (v !== undefined && v !== null) pm.collectionVariables.set('boardCategoryId', String(v)); } catch (e) {}","try {
const v = (Object.keys(j.entities).length); if (v !== undefined && v !== null) pm.collectionVariables.set('documentTypeEntityCount', String(v)); } catch
(e) {}","pm.test('65 record types for an administrator', () => pm.expect(Object.keys(j.entities).length).to.eql(65));","pm.test('19 Standard Lists', () =>
pm.expect(Object.keys(j.lookups).length).to.eql(19));","pm.test('13 calculated views', () =>
pm.expect(Object.keys(j.views).length).to.eql(13));","pm.test('Fields describe their type and requirement', () => { const c =
j.entities.meeting.columns.find(x => x.name === 'meeting_date'); pm.expect(c.kind).to.eql('date'); pm.expect(c.required).to.eql(true);
});","pm.test('Sensitive columns are disabled by policy', () => { const c = j.entities.director.columns.find(x => x.name === 'aadhaar_encrypted');
pm.expect(c.secret_enabled).to.eql(false); });"]}}]},
        {"name":"Dropdown options - board stints of a company","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/options/directorship?
company_id={{companyId}}&limit=20","host":["{{baseUrl}}"],"path":["api","options","directorship"],"query":[{"key":"company_id","value":"{{companyId}}"},
{"key":"limit","value":"20"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Options have id and label', () =>
{ pm.expect(j.options.length).to.be.above(0); pm.expect(j.options[0]).to.have.keys('id', 'label'); });"]}}]},
        {"name":"Dropdown options - search by text","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/options/director?q=joshi","host":
["{{baseUrl}}"],"path":["api","options","director"],"query":[{"key":"q","value":"joshi"}]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","try { const v = (j.options[0].id); if (v !== undefined && v !== null) pm.collectionVariables.set('directorId', String(v)); }
catch (e) {}","pm.test('Search finds the director', () => pm.expect(j.options.some(o => o.label.includes('Joshi'))).to.eql(true));"]}}]},
        {"name":"Dropdown options - fetch by ids","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/options/director?ids=
{{directorId}}","host":["{{baseUrl}}"],"path":["api","options","director"],"query":[{"key":"ids","value":"{{directorId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Exactly the requested record', () => pm.expect(j.options.length).to.eql(1));"]}}]}
      ]
    },
    {
      "name": "03 Companies",
      "item": [
        {"name":"List my companies","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/companies","host":["{{baseUrl}}"],"path":
["api","companies"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Companies carry counts', () => {
pm.expect(j.companies.length).to.be.above(0); pm.expect(j.companies[0]).to.include.keys('name', 'cin', 'directors', 'overdue_filings', 'access_level');
});"]}}]},
        {"name":"Financial years that have data","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/years","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","years"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Years and the current year', () => { pm.expect(j.years).to.be.an('array');
pm.expect(j.current).to.match(/^\\d{4}-\\d{2}$/); });"]}}]},
        {"name":"Overview (tiles, attention, rules, segments)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/overview?fy=2024-25","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","overview"],"query":
[{"key":"fy","value":"2024-25"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Overview has tiles, rules and
segments', () => { pm.expect(j.tiles).to.include.keys('directors', 'meetings_held', 'overdue_filings'); pm.expect(j.rules.total).to.be.above(38);
pm.expect(j.segments.length).to.be.above(3); });"]}}]},
        {"name":"Applicability rules for a year","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/companies/{{companyId}}/rules?
fy=2024-25","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","rules"],"query":[{"key":"fy","value":"2024-25"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.rules.find(r => r.code === 'CSR_SPENDING').rule_id); if (v !== undefined && v !==
null) pm.collectionVariables.set('ruleId', String(v)); } catch (e) {}","pm.test('Rules with evaluations and computed checks', () => {
pm.expect(j.rules.length).to.be.above(38); pm.expect(j.computed).to.have.keys('RESIDENT_DIRECTOR', 'WOMAN_DIRECTOR'); });"]}}]},
        {"name":"Record a rule evaluation (new dated record)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/rule_evaluation","host":["{{baseUrl}}"],"path":["api","entities","rule_evaluation"]},"body":{"mode":"raw","raw":"{\n
\"company_id\": \"{{companyId}}\",\n  \"rule_id\": \"{{ruleId}}\",\n  \"financial_year\": \"2024-25\",\n  \"applies\": true,\n  \"status\": \"pending\",\n
\"notes\": \"Created from Postman\",\n  \"inputs\": {\n    \"net_profit\": \"to be confirmed\"\n  }\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Evaluation time is stamped by the database', () =>
pm.expect(j.row.evaluated_on).to.be.a('string'));"]}}]},
        {"name":"Export one company (Excel)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/export.xlsx","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","export.xlsx"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","pm.test('Is an Excel
workbook', () => pm.expect(pm.response.headers.get('Content-Type')).to.include('spreadsheetml'));"]}}]},
```

```
        {"name":"Export one company (JSON)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/export.json","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","export.json"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Has a sheet per record type', () =>
pm.expect(Object.keys(j.entities).length).to.be.above(40));"]}}]},
        {"name":"Add a company (administrator)","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/companies","host":["
{{baseUrl}}"],"path":["api","companies"]},"body":{"mode":"raw","raw":"{\n  \"cin\": \"{{newCin}}\",\n  \"name\": \"Postman Test Company Private
Limited\",\n  \"date_of_incorporation\": \"2020-04-01\",\n  \"status\": \"active\",\n  \"pan\": \"AAAPA1234A\",\n  \"roc\": \"Mumbai\",\n
\"registered_address\": \"1 Test Road, Mumbai\",\n  \"email\": \"test@example.test\",\n  \"phone\": \"02212345678\",\n  \"is_listed\": false,\n
\"ind_as_applicable\": false\n}","options":{"raw":{"language":"json"}}},"description":"Needs an administrator. The pre-request script invents a unique 21-
character CIN."},"event":[{"listen":"prerequest","script":{"type":"text/javascript","exec":["pm.collectionVariables.set('newCin', 'U' +
String(Date.now()).padStart(20, '0'));"]}},{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () =>
pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.id); if (v !==
undefined && v !== null) pm.collectionVariables.set('newCompanyId', String(v)); } catch (e) {}","try { const v = (j.row.updated_at); if (v !== undefined
&& v !== null) pm.collectionVariables.set('newCompanyVersion', String(v)); } catch (e) {}","pm.test('Creator is given access', () =>
pm.expect(j.row.id).to.be.a('number'));"]}}]},
        {"name":"Edit that company (version-checked)","request":{"method":"PATCH","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company/{{newCompanyId}}","host":["{{baseUrl}}"],"path":["api","entities","company","{{newCompanyId}}"]},"body":
{"mode":"raw","raw":"{\n  \"website\": \"https://example.test\",\n  \"updated_at\": \"{{newCompanyVersion}}\"\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.updated_at); if (v
!== undefined && v !== null) pm.collectionVariables.set('newCompanyVersion', String(v)); } catch (e) {}","pm.test('Website saved', () =>
pm.expect(j.row.website).to.eql('https://example.test'));"]}}]},
        {"name":"Archive that company (nothing is deleted)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company/{{newCompanyId}}/archive","host":["{{baseUrl}}"],"path":["api","entities","company","
{{newCompanyId}}","archive"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Archived flag set', () =>
pm.expect(j.row.is_archived).to.eql(true));"]}}]}
      ]
    },
    {
      "name": "04 Generic records (any table) - full lifecycle",
      "item": [
        {"name":"Create a record (Registrar & Transfer Agent)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta","host":["{{baseUrl}}"],"path":["api","entities","company_rta"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"
{{companyId}}\",\n  \"name\": \"Postman RTA Services\",\n  \"address\": \"Andheri, Mumbai\",\n  \"phone\": \"02200000000\",\n  \"email\":
\"rta@example.test\",\n  \"sebi_reg_no\": \"INR000000001\",\n  \"valid_from\": \"2024-04-01\",\n  \"valid_to\": \"2027-03-31\"\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () =>
pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.id); if (v !==
undefined && v !== null) pm.collectionVariables.set('rtaId', String(v)); } catch (e) {}","try { const v = (j.row.updated_at); if (v !== undefined && v !==
null) pm.collectionVariables.set('rtaVersion', String(v)); } catch (e) {}","try { const v = (j.row.updated_at); if (v !== undefined && v !== null)
pm.collectionVariables.set('rtaVersion0', String(v)); } catch (e) {}","pm.test('Foreign key comes back with a readable label', () =>
pm.expect(j.row._labels.company_id).to.be.a('string'));","pm.test('System columns filled by the database', () => {
pm.expect(j.row.created_at).to.be.a('string'); pm.expect(j.row.is_archived).to.eql(false); });"]}}]},
        {"name":"List with search, sort and paging","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta?company_id=
{{companyId}}&q=Postman&sort=valid_from&desc=true&page=1&size=10","host":["{{baseUrl}}"],"path":["api","entities","company_rta"],"query":
[{"key":"company_id","value":"{{companyId}}"},{"key":"q","value":"Postman"},{"key":"sort","value":"valid_from"},{"key":"desc","value":"true"},
{"key":"page","value":"1"},{"key":"size","value":"10"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200',
() => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Finds the record', () => {
pm.expect(j.total).to.be.above(0); pm.expect(j.rows[0].name).to.include('Postman'); });"]}}]},
        {"name":"List filtered by a column and financial year","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/meeting?
company_id={{companyId}}&fy=2024-25&f_status=held","host":["{{baseUrl}}"],"path":["api","entities","meeting"],"query":[{"key":"company_id","value":"
{{companyId}}"},{"key":"fy","value":"2024-25"},{"key":"f_status","value":"held"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Only held meetings', () => j.rows.forEach(r => pm.expect(r.status).to.eql('held')));"]}}]},
        {"name":"Get one record","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta/{{rtaId}}","host":["
{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status
is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Same record', () =>
pm.expect(j.row.id).to.eql(Number(pm.collectionVariables.get('rtaId'))));"]}}]},
        {"name":"Edit with the version you last saw","request":{"method":"PATCH","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta/{{rtaId}}","host":["{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}"]},"body":{"mode":"raw","raw":"
{\n  \"phone\": \"02211111111\",\n  \"updated_at\": \"{{rtaVersion}}\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","try { const v = (j.row.updated_at); if (v !== undefined && v !== null) pm.collectionVariables.set('rtaVersion', String(v)); }
catch (e) {}","pm.test('Value changed and version moved', () => { pm.expect(j.row.phone).to.eql('02211111111');
pm.expect(j.row.updated_at).to.not.eql(pm.collectionVariables.get('rtaVersion0')); });"]}}]},
        {"name":"Edit with a stale version -> 409","request":{"method":"PATCH","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta/{{rtaId}}","host":["{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}"]},"body":{"mode":"raw","raw":"
{\n  \"phone\": \"02299999999\",\n  \"updated_at\": \"{{rtaVersion0}}\"\n}","options":{"raw":{"language":"json"}}},"description":"Two people editing at
once: the second is told to reload instead of silently overwriting."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 409', () => pm.response.to.have.status(409));","pm.test('Error code is stale', () =>
pm.expect(pm.response.json().error.code).to.eql('stale'));"]}}]},
        {"name":"Validation errors name the fields -> 422","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta","host":["{{baseUrl}}"],"path":["api","entities","company_rta"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"
{{companyId}}\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 422',
() => pm.response.to.have.status(422));","pm.test('Required fields are named', () =>
pm.expect(Object.keys(pm.response.json().error.fields)).to.include.members(['name', 'valid_from']));"]}}]},
        {"name":"A database rule in plain language -> 422","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_business_activity","host":["{{baseUrl}}"],"path":["api","entities","company_business_activity"]},"body":
{"mode":"raw","raw":"{\n  \"company_id\": \"{{companyId}}\",\n  \"description\": \"Too much\",\n  \"turnover_percent\": 140\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 422', () =>
pm.response.to.have.status(422));","pm.test('Friendly message', () => pm.expect(pm.response.json().error.message).to.include('between 0 and 100'));"]}}]},
        {"name":"Record history (who, when, from -> to)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta/{{rtaId}}/history","host":["{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}","history"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Created and changed entries', () => { pm.expect(j.entries.length).to.be.above(1);
pm.expect(j.entries[0].operation).to.eql('UPDATE'); pm.expect(j.entries[0].changes.phone.to).to.eql('02211111111'); });"]}}]},
        {"name":"Archive (hide, never delete)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta/{{rtaId}}/archive","host":["{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}","archive"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Archived', () => pm.expect(j.row.is_archived).to.eql(true));"]}}]},
        {"name":"Archived record appears only when asked","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta?
company_id={{companyId}}&q=Postman&archived=true","host":["{{baseUrl}}"],"path":["api","entities","company_rta"],"query":[{"key":"company_id","value":"
{{companyId}}"},{"key":"q","value":"Postman"},{"key":"archived","value":"true"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Included with archived=true', () => pm.expect(j.rows.some(r => r.is_archived)).to.eql(true));"]}}]},
        {"name":"Restore","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta/{{rtaId}}/restore","host":["
{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}","restore"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Restored', () => pm.expect(j.row.is_archived).to.eql(false));"]}}]},
        {"name":"Export a list as CSV","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta/export.csv?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","entities","company_rta","export.csv"],"query":[{"key":"company_id","value":"
{{companyId}}"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","pm.test('CSV with a header row', () => { pm.expect(pm.response.headers.get('Content-Type')).to.include('text/csv');
```

```
pm.expect(pm.response.text()).to.include('Name'); });"]}}]},
        {"name":"DELETE is not offered -> 405","request":{"method":"DELETE","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/company_rta/{{rtaId}}","host":["{{baseUrl}}"],"path":["api","entities","company_rta","{{rtaId}}"]},"description":"There is no
delete anywhere in the API (and the database forbids it too).","event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is
405', () => pm.response.to.have.status(405));"]}}]},
        {"name":"Unknown record type -> 404","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/not_a_table","host":["
{{baseUrl}}"],"path":["api","entities","not_a_table"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 404', ()
=> pm.response.to.have.status(404));","pm.test('Error code is not_found', () => pm.expect(pm.response.json().error.code).to.eql('not_found'));"]}}]},
        {"name":"Create a meeting (lookup + dates + enum)","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/meeting","host":
["{{baseUrl}}"],"path":["api","entities","meeting"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"{{companyId}}\",\n  \"meeting_type_id\": \"
{{meetingTypeId}}\",\n  \"financial_year\": \"2024-25\",\n  \"meeting_number\": \"PM-1\",\n  \"meeting_date\": \"2025-02-20\",\n  \"start_time\":
\"11:00\",\n  \"end_time\": \"13:00\",\n  \"venue\": \"Postman Boardroom\",\n  \"status\": \"planned\",\n  \"remarks\": \"Created from Postman\",\n
\"members_entitled\": 4\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status
is 201', () => pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.id);
if (v !== undefined && v !== null) pm.collectionVariables.set('meetingId', String(v)); } catch (e) {}","try { const v = (j.row.updated_at); if (v !==
undefined && v !== null) pm.collectionVariables.set('meetingVersion', String(v)); } catch (e) {}","pm.test('Lookup comes back labelled', () =>
pm.expect(j.row._labels.meeting_type_id).to.be.a('string'));"]}}]},
        {"name":"A lookup from the wrong list -> 422","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/meeting","host":["
{{baseUrl}}"],"path":["api","entities","meeting"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"{{companyId}}\",\n  \"meeting_type_id\": \"
{{designationId}}\",\n  \"financial_year\": \"2024-25\",\n  \"meeting_date\": \"2025-02-21\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 422', () => pm.response.to.have.status(422));","pm.test('Names the
field', () => pm.expect(pm.response.json().error.fields).to.have.property('meeting_type_id'));"]}}]},
        {"name":"Mark the meeting as held","request":{"method":"PATCH","header":[],"url":{"raw":"{{baseUrl}}/api/entities/meeting/{{meetingId}}","host":["
{{baseUrl}}"],"path":["api","entities","meeting","{{meetingId}}"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"held\",\n  \"updated_at\": \"
{{meetingVersion}}\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is
200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Status updated', () =>
pm.expect(j.row.status).to.eql('held'));"]}}]},
        {"name":"Add an agenda item to it (child record)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/meeting_agenda_item","host":["{{baseUrl}}"],"path":["api","entities","meeting_agenda_item"]},"body":{"mode":"raw","raw":"{\n
\"meeting_id\": \"{{meetingId}}\",\n  \"item_no\": 1,\n  \"text\": \"Approve the minutes of the previous meeting\"\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () =>
pm.response.to.have.status(201));"]}}]}
      ]
    },
    {
      "name": "05 Directors and KMP register",
      "item": [
        {"name":"Board register (every stint, current and ceased)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/directors-register","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","directors-
register"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let
j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Has current and ceased stints', () => {
pm.expect(j.rows.length).to.be.above(0); pm.expect(j.rows[0]).to.include.keys('full_name', 'din', 'designation', 'start_date', 'is_current'); });"]}}]},
        {"name":"Register - current directors only","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/companies/{{companyId}}/directors-
register?include_ceased=false","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","directors-register"],"query":
[{"key":"include_ceased","value":"false"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Only current', () =>
j.rows.forEach(r => pm.expect(r.is_current).to.eql(true)));"]}}]},
        {"name":"Add a NEW person to the board in one step","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/directors-register","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","directors-
register"]},"body":{"mode":"raw","raw":"{\n  \"director\": {\n    \"full_name\": \"Postman Director\",\n    \"din\": \"{{newDin}}\",\n    \"gender\":
\"F\",\n    \"email\": \"postman.director@example.test\",\n    \"date_of_birth\": \"1985-06-15\",\n    \"nationality\": \"Indian\"\n  },\n
\"directorship\": {\n    \"start_date\": \"2026-04-01\",\n    \"is_independent\": true,\n    \"director_category_id\": \"{{boardCategoryId}}\"\n  },\n
\"designation_id\": \"{{designationId}}\",\n  \"designation_start\": \"2026-04-01\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"prerequest","script":{"type":"text/javascript","exec":["pm.collectionVariables.set('newDin', String(10000000 + (Math.floor(Date.now() / 1000)
% 80000000)));"]}},{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j =
{}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.directorship_id); if (v !== undefined && v !== null)
pm.collectionVariables.set('newDirectorshipId', String(v)); } catch (e) {}","try { const v = (j.row.director_id); if (v !== undefined && v !== null)
pm.collectionVariables.set('newDirectorId', String(v)); } catch (e) {}","pm.test('Person created with stint and designation', () => {
pm.expect(j.person_created).to.eql(true); pm.expect(j.row.designation).to.be.a('string'); pm.expect(j.row.is_current).to.eql(true); });"]}}]},
        {"name":"The same DIN on another company REUSES the person","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{otherCompanyId}}/directors-register","host":["{{baseUrl}}"],"path":["api","companies","{{otherCompanyId}}","directors-
register"]},"body":{"mode":"raw","raw":"{\n  \"director\": {\n    \"full_name\": \"Someone Else\",\n    \"din\": \"{{newDin}}\"\n  },\n  \"directorship\":
{\n    \"start_date\": \"2026-05-01\"\n  }\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Existing person reused (name unchanged)', () => { pm.expect(j.person_created).to.eql(false);
pm.expect(j.row.full_name).to.eql('Postman Director'); });"]}}]},
        {"name":"Overlapping stint is refused -> 409","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/directors-register","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","directors-
register"]},"body":{"mode":"raw","raw":"{\n  \"director\": {\n    \"full_name\": \"Postman Director\",\n    \"din\": \"{{newDin}}\"\n  },\n
\"directorship\": {\n    \"start_date\": \"2026-06-01\"\n  }\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 409', () => pm.response.to.have.status(409));","pm.test('Error code is overlap', () =>
pm.expect(pm.response.json().error.code).to.eql('overlap'));"]}}]},
        {"name":"Missing appointment date -> 422","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/companies/{{companyId}}/directors-
register","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","directors-register"]},"body":{"mode":"raw","raw":"{\n  \"director\": {\n
\"full_name\": \"No Date\"\n  },\n  \"directorship\": {}\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 422', () => pm.response.to.have.status(422));"]}}]}
      ]
    },
    {
      "name": "06 Meetings",
      "item": [
        {"name":"Meetings book for a year","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/companies/{{companyId}}/meetings-book?
fy=2024-25","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","meetings-book"],"query":[{"key":"fy","value":"2024-25"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.meetings.find(m => m.meeting_number === 'BM-1').id); if (v !== undefined && v !==
null) pm.collectionVariables.set('seedMeetingId', String(v)); } catch (e) {}","pm.test('Meetings carry agenda and attendance', () => {
pm.expect(j.meetings.length).to.be.above(0); pm.expect(j.meetings[0]).to.include.keys('agenda', 'attendance', 'meeting_type'); });"]}}]},
        {"name":"One meeting in full (with the board roster)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/meetings/{{seedMeetingId}}/detail","host":["{{baseUrl}}"],"path":["api","meetings","{{seedMeetingId}}","detail"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.meeting.roster[0].directorship_id); if (v !== undefined && v !== null)
pm.collectionVariables.set('rosterDirectorshipId', String(v)); } catch (e) {}","pm.test('Roster = directors on the board that day', () =>
pm.expect(j.meeting.roster.length).to.be.above(0));"]}}]},
        {"name":"Save attendance by roster (create or update, never duplicate)","request":{"method":"PUT","header":[],"url":{"raw":"
{{baseUrl}}/api/meetings/{{seedMeetingId}}/attendance","host":["{{baseUrl}}"],"path":["api","meetings","{{seedMeetingId}}","attendance"]},"body":
{"mode":"raw","raw":"{\n  \"rows\": [\n    {\n      \"directorship_id\": \"{{rosterDirectorshipId}}\",\n      \"attended\": true,\n      \"mode\":
\"Physical\"\n    }\n  ]\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status
is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Saved', () => {
pm.expect(j.saved).to.eql(1); pm.expect(j.attended).to.be.above(0); });"]}}]}
      ]
    },
    {
      "name": "07 Calculated values (database views)",
      "item": [
```

```
        {"name":"v_current_directors","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_current_directors?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_current_directors"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_director_history","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_director_history?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_director_history"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_director_age","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_director_age?company_id={{companyId}}","host":
["{{baseUrl}}"],"path":["api","calc","v_director_age"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_shareholding_percent","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_shareholding_percent?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_shareholding_percent"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_top10_shareholders","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_top10_shareholders?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_top10_shareholders"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_company_shareholder_counts","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_company_shareholder_counts?
company_id={{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_company_shareholder_counts"],"query":[{"key":"company_id","value":"
{{companyId}}"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => {
pm.expect(j.columns).to.be.an('array').and.not.empty; pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_director_attendance_summary","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_director_attendance_summary?
company_id={{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_director_attendance_summary"],"query":[{"key":"company_id","value":"
{{companyId}}"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => {
pm.expect(j.columns).to.be.an('array').and.not.empty; pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_committee_attendance_percent","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_committee_attendance_percent?
company_id={{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_committee_attendance_percent"],"query":[{"key":"company_id","value":"
{{companyId}}"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => {
pm.expect(j.columns).to.be.an('array').and.not.empty; pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_independent_director_years","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_independent_director_years?
company_id={{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_independent_director_years"],"query":[{"key":"company_id","value":"
{{companyId}}"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => {
pm.expect(j.columns).to.be.an('array').and.not.empty; pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_s186_running_total","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_s186_running_total?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_s186_running_total"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_filings_overdue","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_filings_overdue?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_filings_overdue"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_rule_resident_director","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_rule_resident_director?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_rule_resident_director"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"v_rule_woman_director","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/v_rule_woman_director?company_id=
{{companyId}}","host":["{{baseUrl}}"],"path":["api","calc","v_rule_woman_director"],"query":[{"key":"company_id","value":"{{companyId}}"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Returns columns and rows', () => { pm.expect(j.columns).to.be.an('array').and.not.empty;
pm.expect(j.rows).to.be.an('array'); });"]}}]},
        {"name":"Unknown view -> 404","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/calc/not_a_view","host":["{{baseUrl}}"],"path":
["api","calc","not_a_view"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 404', () =>
pm.response.to.have.status(404));"]}}]}
      ],
      "description": "The database views, filtered by the API to the companies the user may see."
    },
    {
      "name": "08 Activities and tasks",
      "item": [
        {"name":"List activity templates","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/activities/templates","host":["
{{baseUrl}}"],"path":["api","activities","templates"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', ()
=> pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.templates[0].id); if
(v !== undefined && v !== null) pm.collectionVariables.set('templateId', String(v)); } catch (e) {}","try { const v = (j.templates[0].code); if (v !==
undefined && v !== null) pm.collectionVariables.set('templateCode', String(v)); } catch (e) {}","pm.test('Board Meeting template, 9 steps', () => {
pm.expect(j.templates[0].code).to.eql('BOARD_MEETING'); pm.expect(j.templates[0].steps).to.eql(9); });"]}}]},
        {"name":"Template detail (steps, bindings, evidence)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/templates/{{templateId}}","host":["{{baseUrl}}"],"path":["api","activities","templates","{{templateId}}"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('9 ordered steps with bindings', () => { pm.expect(j.steps.length).to.eql(9);
pm.expect(j.steps[0].bindings.length).to.be.above(0); });"]}}]},
        {"name":"Template as one JSON document (the shape a future MCP tool will use)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/templates/{{templateCode}}/json","host":["{{baseUrl}}"],"path":["api","activities","templates","
{{templateCode}}","json"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Has template and steps', () => {
pm.expect(j.template.code).to.eql('BOARD_MEETING'); pm.expect(j.steps.length).to.eql(9); });"]}}]},
        {"name":"Start without required fields -> 422","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/activities/instances","host":
["{{baseUrl}}"],"path":["api","activities","instances"]},"body":{"mode":"raw","raw":"{\n  \"template_code\": \"BOARD_MEETING\",\n  \"company_id\": \"
{{companyId}}\",\n  \"params\": {\n    \"venue\": \"x\"\n  }\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 422', () => pm.response.to.have.status(422));","pm.test('Missing fields named', () =>
pm.expect(Object.keys(pm.response.json().error.fields)).to.include('params.meeting_date'));"]}}]},
        {"name":"Start a Board Meeting activity (creates 9 tasks with due dates)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/instances","host":["{{baseUrl}}"],"path":["api","activities","instances"]},"body":{"mode":"raw","raw":"{\n  \"template_code\":
\"BOARD_MEETING\",\n  \"company_id\": \"{{companyId}}\",\n  \"params\": {\n    \"meeting_date\": \"2026-12-10\",\n    \"start_time\": \"11:00\",\n
\"venue\": \"5th Floor, Nariman Point, Mumbai\",\n    \"venue_mode\": \"PHYSICAL\",\n    \"agenda_items\": [\n      \"Approve the accounts\",\n
\"Appoint the auditor\"\n    ]\n  }\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try {
const v = (j.instance.id); if (v !== undefined && v !== null) pm.collectionVariables.set('instanceId', String(v)); } catch (e) {}","try { const v =
(j.tasks.find(t => t.step.seq === 1).id); if (v !== undefined && v !== null) pm.collectionVariables.set('taskId1', String(v)); } catch (e) {}","try {
const v = (j.tasks.find(t => t.step.seq === 2).id); if (v !== undefined && v !== null) pm.collectionVariables.set('taskId2', String(v)); } catch (e)
```

```
{}","try { const v = (j.tasks.find(t => t.step.seq === 3).id); if (v !== undefined && v !== null) pm.collectionVariables.set('taskId3', String(v)); }
catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 4).id); if (v !== undefined && v !== null) pm.collectionVariables.set('taskId4',
String(v)); } catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 5).id); if (v !== undefined && v !== null)
pm.collectionVariables.set('taskId5', String(v)); } catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 6).id); if (v !== undefined && v !==
null) pm.collectionVariables.set('taskId6', String(v)); } catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 7).id); if (v !== undefined &&
v !== null) pm.collectionVariables.set('taskId7', String(v)); } catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 8).id); if (v !==
undefined && v !== null) pm.collectionVariables.set('taskId8', String(v)); } catch (e) {}","try { const v = (j.tasks.find(t => t.step.seq === 9).id); if
(v !== undefined && v !== null) pm.collectionVariables.set('taskId9', String(v)); } catch (e) {}","pm.test('Nine tasks, due dates worked out', () => {
pm.expect(j.tasks.length).to.eql(9); const t = j.tasks.find(x => x.step.step_key === 'DRAFT_NOTICE'); pm.expect(t.due_date).to.eql('2026-12-03'); const m
= j.tasks.find(x => x.step.step_key === 'SIGN_MINUTES'); pm.expect(m.due_date).to.eql('2027-01-09'); });"]}}]},
        {"name":"This company's activities","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/activities","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","activities"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Includes progress counts', () => { pm.expect(j.instances.length).to.be.above(0);
pm.expect(j.instances[0]).to.include.keys('tasks', 'done', 'overdue'); });"]}}]},
        {"name":"One activity with tasks, evidence and confirmations","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/instances/{{instanceId}}","host":["{{baseUrl}}"],"path":["api","activities","instances","{{instanceId}}"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Tasks carry step details and blockers', () => {
pm.expect(j.tasks[1].blocked_by.length).to.be.above(0); pm.expect(j.tasks[0].step).to.include.keys('title', 'bindings', 'evidence'); });"]}}]},
        {"name":"Complete step 2 first -> 409 blocked","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId2}}/status","host":["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId2}}","status"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":{"language":"json"}}},"description":"A step cannot be completed before the step
it depends on."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 409', () =>
pm.response.to.have.status(409));","pm.test('Error code is blocked', () => pm.expect(pm.response.json().error.code).to.eql('blocked'));"]}}]},
        {"name":"Step 1 -> in progress","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/activities/tasks/{{taskId1}}/status","host":
["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId1}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"in_progress\"\n}","options":
{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Instance still active', () =>
pm.expect(j.instance.status).to.eql('active'));"]}}]},
        {"name":"Step 1 -> completed","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/activities/tasks/{{taskId1}}/status","host":["
{{baseUrl}}"],"path":["api","activities","tasks","{{taskId1}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"completed\",\n  \"notes\": \"Date
agreed with the Chairman\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Completed by me', () => { const t = j.tasks.find(x => x.step.seq === 1); pm.expect(t.status).to.eql('completed');
pm.expect(t.completed_at).to.be.a('string'); });"]}}]},
        {"name":"Step 2 -> completed","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/activities/tasks/{{taskId2}}/status","host":["
{{baseUrl}}"],"path":["api","activities","tasks","{{taskId2}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));"]}}]},
        {"name":"Step 3 -> completed (warns about missing evidence)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId3}}/status","host":["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId3}}","status"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Warning about mandatory evidence', () => pm.expect(j.warnings.join(' ')).to.include('No evidence'));"]}}]},
        {"name":"Step 4 needs a confirmation -> 409","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId4}}/status","host":["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId4}}","status"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 409', () => pm.response.to.have.status(409));","pm.test('Error code is needs_confirmation', () =>
pm.expect(pm.response.json().error.code).to.eql('needs_confirmation'));"]}}]},
        {"name":"Request the confirmation","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId4}}/confirmations","host":["{{baseUrl}}"],"path":["api","activities","tasks","
{{taskId4}}","confirmations"]},"body":{"mode":"raw","raw":"{}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","try { const v = (j.confirmation.id); if (v !== undefined && v !== null) pm.collectionVariables.set('confirmationId', String(v));
} catch (e) {}","pm.test('Pending', () => pm.expect(j.confirmation.status).to.eql('pending'));"]}}]},
        {"name":"Confirm it (an administrator may confirm their own request)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/confirmations/{{confirmationId}}/confirm","host":["{{baseUrl}}"],"path":["api","confirmations","
{{confirmationId}}","confirm"]},"description":"For other users a *different* person must confirm: sign in as the manager and run this request."},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Confirmed by a named person', () => { pm.expect(j.confirmation.status).to.eql('confirmed');
pm.expect(j.confirmation.confirmed_by).to.be.a('number'); });"]}}]},
        {"name":"Step 4 -> completed (now allowed)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId4}}/status","host":["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId4}}","status"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));"]}}]},
        {"name":"Skip step 5","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/activities/tasks/{{taskId5}}/status","host":["
{{baseUrl}}"],"path":["api","activities","tasks","{{taskId5}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"skipped\"\n}","options":{"raw":
{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));"]}}]},
        {"name":"Cancel the activity","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/instances/{{instanceId}}/status","host":["{{baseUrl}}"],"path":["api","activities","instances","
{{instanceId}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"cancelled\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Cancelled', () => pm.expect(j.instance.status).to.eql('cancelled'));"]}}]},
        {"name":"Nothing moves while cancelled -> 409","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/tasks/{{taskId6}}/status","host":["{{baseUrl}}"],"path":["api","activities","tasks","{{taskId6}}","status"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"completed\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 409', () => pm.response.to.have.status(409));"]}}]},
        {"name":"Reopen the activity","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/instances/{{instanceId}}/status","host":["{{baseUrl}}"],"path":["api","activities","instances","
{{instanceId}}","status"]},"body":{"mode":"raw","raw":"{\n  \"status\": \"active\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Active again', () => pm.expect(j.instance.status).to.eql('active'));"]}}]},
        {"name":"Edit a task note (the only directly editable task field)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/task_instance/{{taskId1}}","host":["{{baseUrl}}"],"path":["api","entities","task_instance","{{taskId1}}"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.row.updated_at); if (v !== undefined && v !== null)
pm.collectionVariables.set('taskVersion', String(v)); } catch (e) {}"]}}]},
        {"name":"Save the note","request":{"method":"PATCH","header":[],"url":{"raw":"{{baseUrl}}/api/entities/task_instance/{{taskId1}}","host":["
{{baseUrl}}"],"path":["api","entities","task_instance","{{taskId1}}"]},"body":{"mode":"raw","raw":"{\n  \"notes\": \"Edited from Postman\",\n
\"updated_at\": \"{{taskVersion}}\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Note saved', () => pm.expect(j.row.notes).to.eql('Edited from Postman'));"]}}]},
        {"name":"Task status cannot be PATCHed directly -> 422","request":{"method":"PATCH","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/task_instance/{{taskId1}}","host":["{{baseUrl}}"],"path":["api","entities","task_instance","{{taskId1}}"]},"body":
{"mode":"raw","raw":"{\n  \"status\": \"not_started\",\n  \"updated_at\": \"{{taskVersion}}\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 422', () => pm.response.to.have.status(422));"]}}]},
        {"name":"Link a record to a task","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/task_record_link","host":["
{{baseUrl}}"],"path":["api","entities","task_record_link"]},"body":{"mode":"raw","raw":"{\n  \"task_instance_id\": \"{{taskId1}}\",\n  \"entity_type\":
\"meeting\",\n  \"entity_id\": \"{{seedMeetingId}}\",\n  \"mode\": \"write\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Linked record is labelled', () =>
pm.expect(j.row._labels.entity_id).to.be.a('string'));"]}}]}
```

```
      ]
    },
    {
      "name": "09 Documents and evidence",
      "item": [
        {"name":"Upload a document and link it to a meeting","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/documents/upload","host":["{{baseUrl}}"],"path":["api","documents","upload"]},"body":{"mode":"formdata","formdata":
[{"key":"company_id","value":"{{companyId}}","type":"text"},{"key":"file","type":"file","src":"sample_notice.pdf"},
{"key":"entity_type","value":"meeting","type":"text"},{"key":"entity_id","value":"{{seedMeetingId}}","type":"text"},{"key":"note","value":"Uploaded from
Postman","type":"text"}]},"description":"In Postman choose any small PDF/Word/Excel/image file for the `file` field (the collection expects
`sample_notice.pdf` next to it; Newman needs `--working-dir`). Types: pdf, doc(x), xls(x), csv, txt, rtf, odt, ods, images. Maximum 25 MB."},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.document.id); if (v !== undefined && v !== null)
pm.collectionVariables.set('documentId', String(v)); } catch (e) {}","try { const v = (j.document.sha256); if (v !== undefined && v !== null)
pm.collectionVariables.set('documentSha', String(v)); } catch (e) {}","pm.test('Fingerprinted and versioned', () => {
pm.expect(j.document.sha256).to.have.lengthOf(64); pm.expect(j.document.version).to.be.at.least(1);
pm.expect(j.evidence_link.entity_type).to.eql('meeting'); });"]}}]},
        {"name":"Download it again (integrity re-checked)","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/documents/{{documentId}}/download","host":["{{baseUrl}}"],"path":["api","documents","{{documentId}}","download"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","pm.test('Sent as an
attachment', () => pm.expect(pm.response.headers.get('Content-Disposition')).to.include('attachment'));"]}}]},
        {"name":"The document's record","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/document/{{documentId}}","host":["
{{baseUrl}}"],"path":["api","entities","document","{{documentId}}"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */
}","pm.test('Fingerprint stored', () => pm.expect(j.row.sha256).to.eql(pm.collectionVariables.get('documentSha')));"]}}]},
        {"name":"Upload with no file -> 422","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/documents/upload","host":["
{{baseUrl}}"],"path":["api","documents","upload"]},"body":{"mode":"formdata","formdata":[{"key":"company_id","value":"
{{companyId}}","type":"text"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 422', () =>
pm.response.to.have.status(422));"]}}]}
      ]
    },
    {
      "name": "10 Audit trail",
      "item": [
        {"name":"Activity log for a company","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/audit?company_id=
{{companyId}}&limit=20","host":["{{baseUrl}}"],"path":["api","audit"],"query":[{"key":"company_id","value":"{{companyId}}"},
{"key":"limit","value":"20"}]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Entries say who, what and when',
() => { pm.expect(j.entries.length).to.be.above(0); pm.expect(j.entries[0]).to.include.keys('at', 'actor', 'operation', 'table', 'changes'); });"]}}]},
        {"name":"Everything I may see (filtered by table)","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/audit?
table=meeting&limit=10","host":["{{baseUrl}}"],"path":["api","audit"],"query":[{"key":"table","value":"meeting"},{"key":"limit","value":"10"}]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j =
pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Only meeting entries', () => j.entries.forEach(e =>
pm.expect(e.table).to.eql('meeting')));"]}}]}
      ]
    },
    {
      "name": "11 Administration (administrator only)",
      "item": [
        {"name":"List users","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/app_user?q=client","host":["
{{baseUrl}}"],"path":["api","entities","app_user"],"query":[{"key":"q","value":"client"}]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","try { const v = (j.rows[0].id); if (v !== undefined && v !== null) pm.collectionVariables.set('clientUserId', String(v)); } catch
(e) {}","pm.test('Client user found', () => pm.expect(j.rows[0].email).to.include('client'));"]}}]},
        {"name":"Set a user's sign-in password","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/admin/users/{{clientUserId}}/password","host":["{{baseUrl}}"],"path":["api","admin","users","{{clientUserId}}","password"]},"body":
{"mode":"raw","raw":"{\n  \"new_password\": \"{{clientPassword}}\"\n}","options":{"raw":{"language":"json"}}},"description":"Re-sets the client demo
user's password to the value in your environment (harmless)."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is
200', () => pm.response.to.have.status(200));"]}}]},
        {"name":"Create a user","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/app_user","host":["{{baseUrl}}"],"path":
["api","entities","app_user"]},"body":{"mode":"raw","raw":"{\n  \"email\": \"{{newUserEmail}}\",\n  \"name\": \"Postman User\",\n  \"is_active\":
true\n}","options":{"raw":{"language":"json"}}},"event":[{"listen":"prerequest","script":{"type":"text/javascript","exec":
["pm.collectionVariables.set('newUserEmail', 'postman.' + Date.now() + '@example.test');"]}},{"listen":"test","script":{"type":"text/javascript","exec":
["pm.test('Status is 201', () => pm.response.to.have.status(201));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try {
const v = (j.row.id); if (v !== undefined && v !== null) pm.collectionVariables.set('newUserId', String(v)); } catch (e) {}"]}}]},
        {"name":"Grant that user read access to a company","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/user_company_access","host":["{{baseUrl}}"],"path":["api","entities","user_company_access"]},"body":{"mode":"raw","raw":"{\n
\"user_id\": \"{{newUserId}}\",\n  \"company_id\": \"{{companyId}}\",\n  \"access_level\": \"read\"\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 201', () => pm.response.to.have.status(201));"]}}]},
        {"name":"Archive the user (users are never deleted)","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/app_user/{{newUserId}}/archive","host":["{{baseUrl}}"],"path":["api","entities","app_user","{{newUserId}}","archive"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));"]}}]},
        {"name":"List Standard List values","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/lookup_value?size=5","host":["
{{baseUrl}}"],"path":["api","entities","lookup_value"],"query":[{"key":"size","value":"5"}]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('102+ values', () => pm.expect(j.total).to.be.at.least(102));"]}}]}
      ]
    },
    {
      "name": "12 Isolation checks (signs in as the read-only client, then back as admin)",
      "item": [
        {"name":"Login as the read-only client (Nexatech only)","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/login","host":["
{{baseUrl}}"],"path":["api","auth","login"]},"body":{"mode":"raw","raw":"{\n  \"email\": \"{{clientEmail}}\",\n  \"password\": \"
{{clientPassword}}\"\n}","options":{"raw":{"language":"json"}}},"description":"Signs in. The test script stores the returned CSRF token in the collection
variable `csrf`; the collection's pre-request script then sends it as the `X-CSRF-Token` header on every change. The session cookie is kept by Postman's
cookie jar."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let
j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.csrf); if (v !== undefined && v !== null)
pm.collectionVariables.set('csrf', String(v)); } catch (e) {}","pm.test('Returns the user and a CSRF token', () => {
pm.expect(j.csrf).to.be.a('string').and.not.empty; pm.expect(j.user.email).to.be.a('string'); });","pm.test('is_admin is false', () =>
pm.expect(j.user.is_admin).to.eql(false));"]}}]},
        {"name":"Client sees only their company","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/companies","host":["
{{baseUrl}}"],"path":["api","companies"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () =>
pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","pm.test('Exactly one company', () =>
pm.expect(j.companies.length).to.eql(1));","pm.test('It is the client company', () =>
pm.expect(String(j.companies[0].id)).to.eql(pm.variables.get('clientCompanyId')));"]}}]},
        {"name":"Another company's overview -> 404","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/overview","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","overview"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 404', () => pm.response.to.have.status(404));"]}}]},
        {"name":"Another company's export -> 404","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/companies/{{companyId}}/export.xlsx","host":["{{baseUrl}}"],"path":["api","companies","{{companyId}}","export.xlsx"]}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 404', () => pm.response.to.have.status(404));"]}}]},
        {"name":"Administration tables are hidden -> 404","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/entities/app_user","host":["
{{baseUrl}}"],"path":["api","entities","app_user"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 404', () =>
pm.response.to.have.status(404));"]}}]},
```

```
        {"name":"A read-only user cannot write -> 403","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/entities/company_rta","host":
["{{baseUrl}}"],"path":["api","entities","company_rta"]},"body":{"mode":"raw","raw":"{\n  \"company_id\": \"{{clientCompanyId}}\",\n  \"name\": \"Not
allowed\",\n  \"valid_from\": \"2024-04-01\"\n}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 403', () => pm.response.to.have.status(403));","pm.test('Error code is forbidden', () =>
pm.expect(pm.response.json().error.code).to.eql('forbidden'));"]}}]},
        {"name":"Calculated views are narrowed to their company","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/calc/v_current_directors","host":["{{baseUrl}}"],"path":["api","calc","v_current_directors"]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Only their company rows', () => j.rows.forEach(r =>
pm.expect(String(r.company_id)).to.eql(pm.variables.get('clientCompanyId'))));"]}}]},
        {"name":"People are limited to those linked to their company","request":{"method":"GET","header":[],"url":{"raw":"
{{baseUrl}}/api/entities/director","host":["{{baseUrl}}"],"path":["api","entities","director"]}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Fewer than the 12 people the administrator sees', () => pm.expect(j.total).to.be.below(12));"]}}]},
        {"name":"Client cannot start an activity -> 403","request":{"method":"POST","header":[],"url":{"raw":"
{{baseUrl}}/api/activities/instances","host":["{{baseUrl}}"],"path":["api","activities","instances"]},"body":{"mode":"raw","raw":"{\n  \"template_code\":
\"BOARD_MEETING\",\n  \"company_id\": \"{{clientCompanyId}}\",\n  \"params\": {}\n}","options":{"raw":{"language":"json"}}}},"event":
[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 403', () => pm.response.to.have.status(403));"]}}]},
        {"name":"Sign out the client","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/logout","host":["{{baseUrl}}"],"path":
["api","auth","logout"]},"body":{"mode":"raw","raw":"{}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));"]}}]},
        {"name":"Sign back in as administrator","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/login","host":["
{{baseUrl}}"],"path":["api","auth","login"]},"body":{"mode":"raw","raw":"{\n  \"email\": \"{{adminEmail}}\",\n  \"password\": \"
{{adminPassword}}\"\n}","options":{"raw":{"language":"json"}}},"description":"Signs in. The test script stores the returned CSRF token in the collection
variable `csrf`; the collection's pre-request script then sends it as the `X-CSRF-Token` header on every change. The session cookie is kept by Postman's
cookie jar."},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let
j = {}; try { j = pm.response.json(); } catch (e) { /* not JSON */ }","try { const v = (j.csrf); if (v !== undefined && v !== null)
pm.collectionVariables.set('csrf', String(v)); } catch (e) {}","pm.test('Returns the user and a CSRF token', () => {
pm.expect(j.csrf).to.be.a('string').and.not.empty; pm.expect(j.user.email).to.be.a('string'); });","pm.test('is_admin is true', () =>
pm.expect(j.user.is_admin).to.eql(true));"]}}]}
      ]
    },
    {
      "name": "13 Sign out",
      "item": [
        {"name":"Logout","request":{"method":"POST","header":[],"url":{"raw":"{{baseUrl}}/api/auth/logout","host":["{{baseUrl}}"],"path":
["api","auth","logout"]},"body":{"mode":"raw","raw":"{}","options":{"raw":{"language":"json"}}}},"event":[{"listen":"test","script":
{"type":"text/javascript","exec":["pm.test('Status is 200', () => pm.response.to.have.status(200));","let j = {}; try { j = pm.response.json(); } catch
(e) { /* not JSON */ }","pm.test('Signed out', () => pm.expect(j.ok).to.eql(true));"]}}]},
        {"name":"Session is gone -> 401","request":{"method":"GET","header":[],"url":{"raw":"{{baseUrl}}/api/auth/me","host":["{{baseUrl}}"],"path":
["api","auth","me"]}},"event":[{"listen":"test","script":{"type":"text/javascript","exec":["pm.test('Status is 401', () =>
pm.response.to.have.status(401));"]}}]}
      ]
    }
  ]
}
```

# Appendix C — Glossary

| Term | Meaning |
|---|---|
| **Activity / task** | A compliance recipe (e.g. Board Meeting) and the dated to-do items created from it |
| **Archive** | Hide a record without deleting it |
| **Audit log** | Append-only record of who changed what and when |
| **CIN / DIN / DSC** | Company Identification Number / Director Identification Number / Digital Signature Certificate |
| **CSRF** | Cross-site request forgery; blocked by the `X-CSRF-Token` header |
| **Lookup / Standard List** | A managed dropdown list stored in the database (e.g. designations) |
| **Optimistic lock** | Edit carries the version you saw; stale edits are refused |
| **RLS** | Row-level security: the database itself hides other companies' rows |
| **Scope** | The API's rule for which rows a user may read or write |
| **Stint** | One period of a person serving on a company's board |